RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

UNIVERSITÉ MARIE & LOUIS PASTEUR • SUP-FC

INFORMATIQUE
2026 - 2027

Evaluation de Programmes

Nicolas VACELET

SUP-FC
Accompagnement pédagogique
Formations à distance - Certifications

UNIVERSITÉ MARIE & LOUIS PASTEUR

Évaluation de programmes

Nicolas VACELET
Responsable de module
Université Marie et Louis Pasteur

Avant-propos

Si vous lisez ces lignes, c'est que vous êtes sur le point d'embarquer pour un voyage au cœur de ce que les ordinateurs peuvent et ne peuvent pas faire.

Ce cours est conçu pour être lu et compris en autonomie. Il y a un début, un milieu et une fin (spoiler : la fin vaut un million de dollars).

Ce document s'appuie initialement sur une partie de cours des années précédentes donnés à l'Université Marie et Louis Pasteur. Il a été complètement remanié pour être plus accessible, retravaillé et condensé afin d'être plus interactif et davantage en accord avec la maquette actuelle.

Bonne lecture !

Comment lire ce cours ?

- **Lisez dans l'ordre.** Chaque chapitre s'appuie sur le précédent. Sautez des chapitres si vous le voulez, mais vous allez être perdu.
- **Faites les exercices et les quiz.**
- **Prenez votre temps.** Certains concepts sont subtils. Si vous ne comprenez pas du premier coup, relisez, c'est normal. N'hésitez pas à aller fouiner ailleurs, dans d'autres cours.

Les petites boîtes colorées

Tout au long de ce cours, vous trouverez des encadrés spéciaux :

Pause café

Les **pauses café** proposent des analogies ou des moments de détente pour mieux digérer un concept difficile.

À vous de jouer !

Les **interpellations** vous posent une question directe. Essayez d'y répondre *avant* de lire la suite !

Quiz rapide

Les **quiz rapides** testent votre compréhension.

Question de réflexion

Les **questions de réflexion** vous invitent à aller plus loin.

Un peu d'histoire

Les encadrés **historiques** racontent les personnes derrière les théorèmes. Parce que les maths et l'informatique, c'est aussi des histoires humaines.

Sommaire

Avant-propos **1**

1 Introduction **6**
1.1 Problème . . . 6
1.1.1 Problèmes de décision et problèmes d'optimisation . . . 7
1.2 Algorithme . . . 8
1.3 Existe-t-il un algorithme pour chaque problème ? . . . 10
1.3.1 Un exemple concret : le problème de pavage . . . 10
1.3.2 Que signifie "aucun algorithme n'existe" ? . . . 11
1.3.3 Décidable, indécidable . . . 12
1.4 Tous les algorithmes sont-ils utilisables ? . . . 12
1.4.1 Problèmes traitables et non traitables . . . 13
1.5 Complexité et algorithmique . . . 13
1.6 La grande carte de l'informatique théorique . . . 14

2 Les machines de Turing **16**
2.1 Le vocabulaire de base . . . 16
2.2 Présentation intuitive . . . 17
2.3 Définition formelle . . . 18
2.4 Configurations et calculs . . . 19
2.4.1 Configuration . . . 19
2.4.2 Étape de calcul . . . 20
2.4.3 Exemple de calcul complet . . . 20
2.5 Mots et langages reconnus . . . 21
2.6 Langages et fonctions décidables . . . 22
2.6.1 Fonctions décidables . . . 22
2.7 Variations sur la machine de Turing . . . 23
2.7.1 Machine à plusieurs rubans . . . 23
2.7.2 Machine non déterministe . . . 23
2.7.3 La thèse de Church-Turing . . . 24
2.8 Machines de Turing comme énumérateurs . . . 24
2.9 La machine de Turing universelle . . . 25
2.9.1 L'idée clé : un programme est une donnée . . . 25
2.9.2 Définition formelle . . . 26
2.9.3 Importance conceptuelle . . . 27
2.10 La hiérarchie de Chomsky . . . 27
2.10.1 Langages réguliers (Type 3) . . . 28
2.10.2 Langages algébriques / hors-contexte (Type 2) . . . 28
2.10.3 Langages contextuels (Type 1) . . . 29
2.10.4 Les machines de Turing . . . 29
2.11 Récapitulatif . . . 29

3 Décidabilité et indécidabilité **31**
3.1 Rappel : la thèse de Church-Turing . . . 31
3.2 Langages et problèmes décidables . . . 31
3.2.1 Propriétés de fermeture des langages décidables . . . 32
3.3 Problèmes indécidables . . . 33
3.3.1 Codage des machines de Turing . . . 33
3.3.2 Le langage diagonal L_d . . . 34
3.4 Réductions . . . 35
3.4.1 Le problème de l'acceptation (problème universel) . . . 36
3.4.2 Le problème de l'arrêt . . . 37
3.5 Semi-décidabilité . . . 39
3.5.1 La classe co-RE et la hiérarchie R / RE / co-RE . . . 40
3.6 Le théorème de Rice . . . 41
3.7 Bilan . . . 42
3.8 Exercices . . . 43

4 Classes de complexité (Tous les algorithmes ne se valent pas) **45**
4.1 Pourquoi mesurer la complexité ? . . . 45
4.2 Calculer la complexité . . . 46
4.3 La notation grand-O . . . 46
4.3.1 Règles de comptage . . . 47
4.3.2 Comparaison d'algorithmes . . . 48
4.4 Complexité d'une machine de Turing . . . 49
4.4.1 Compter les étapes et les cases . . . 49
4.4.2 Et pour les machines non déterministes ? . . . 50
4.5 Les grandes classes de complexité . . . 50
4.5.1 Les quatre familles . . . 50
4.5.2 La relation entre déterministe et non déterministe . . . 51
4.5.3 $\mathcal{P}$ et $\mathcal{NP}$ . . . 51
4.5.4 co-$\mathcal{NP}$ . . . 52
4.5.5 Pour bien comprendre : un exemple avec le Sudoku . . . 53
4.5.6 La thèse de l'invariance . . . 53
4.6 La hiérarchie des vitesses . . . 53
4.6.1 Le théorème de hiérarchie temporelle . . . 54
4.7 Le théorème de Savitch : espace déterministe vs non déterministe . . . 54
4.7.1 Intuition : la technique de bissection . . . 55
4.7.2 Conséquences et hiérarchie complète . . . 56
4.8 Exercices . . . 56
4.9 Résumé : la grande image . . . 57

5 $\mathcal{P}$ vs $\mathcal{NP}$ **60**
5.1 Retour sur $\mathcal{P}$ et $\mathcal{NP}$ . . . 60
5.1.1 Pourquoi c'est important . . . 61
5.2 Les problèmes $\mathcal{NP}$-complets . . . 62
5.2.1 Transformations polynomiales (réductions) . . . 62
5.2.2 Définition formelle de $\mathcal{NP}$-complet . . . 63
5.2.3 $\mathcal{NP}$-difficile vs $\mathcal{NP}$-complet . . . 63
5.3 SAT, Le premier problème $\mathcal{NP}$-complet . . . 64
5.3.1 Rappels de logique . . . 64

5.3.2 Forme normale conjonctive . . . 64
5.3.3 Le problème SAT . . . 65
5.3.4 Le théorème de Cook-Levin (1971) . . . 65
5.3.5 3-SAT : la variante canonique . . . 66
5.3.6 Pourquoi SAT est dans $\mathcal{NP}$ . . . 67
5.4 La galaxie des problèmes $\mathcal{NP}$-complets . . . 67
5.4.1 Le problème du voyageur de commerce . . . 67
5.4.2 Le problème du sac à dos . . . 68
5.4.3 La coloration de graphe . . . 68
5.4.4 Le problème de la clique . . . 68
5.4.5 Comment prouver qu'un problème est $\mathcal{NP}$-complet . . . 68
5.4.6 Exemple complet de réduction : Clique $\leq_p$ Ensemble Indépendant . 69
5.5 Les stratégies face à la $\mathcal{NP}$-complétude . . . 70
5.6 Au-delà de $\mathcal{NP}$: un aperçu . . . 70
5.6.1 Problèmes $\mathcal{NP}$-intermédiaires : le théorème de Ladner . . . 71
5.6.2 La hiérarchie polynomiale . . . 71
5.7 L'état de la question : où en est-on ? . . . 72
5.8 Quiz final . . . 73
5.9 Exercices . . . 74
5.10 Conclusion du cours . . . 76

1 Introduction

Un peu d'histoire

Nous sommes dans les années 1930. Le monde est en crise, mais dans les couloirs des universités, des logiciens brillants se posent une question étrange : *qu'est-ce que calculer ?*
Kurt Gödel, Alonzo Church et Alan Turing vont, chacun à leur manière, tenter d'y répondre. Leurs travaux vont littéralement **inventer** l'informatique théorique, cette science qui n'existe pas encore. Ces gens ont défini les limites fondamentales de ce que les ordinateurs peuvent faire alors qu'il n'existait pas encore d'ordinateur.

À retenir

Objectifs du chapitre :

- Comprendre ce qu'est un **problème** en informatique.
- Découvrir la notion d'**algorithme** et ses limites.
- Apprendre que certains problèmes sont **indécidables**.
- Avoir un premier aperçu de la **complexité** et de la question P vs NP.

1.1 Problème

Commençons par le commencement. Il faut définir les termes. En informatique, tout part d'un **problème**. Un problème au sens informatique, c'est quoi ?

À retenir

Un **problème** en informatique est constitué de :

- des **données** sous une certaine forme,
- une **question** portant sur ces données.

ET c'est tout. Mais cette simplicité est trompeuse. Regardons quelques exemples pour bien comprendre :

Problème 1
Exemple 1.1
Données : *250 gr. de farine, 0,5 litre de lait, 2 œufs, 2 gr. de sel.*
Question : *Comment faire une pâte à crêpes ?*

Problème 2

Données : *Une voiture accidentée.*
Question : *Est-elle réparable ?*

Problème 3

Données : $n \in \mathbb{N}$
Question : *n est-il divisible par 10 ?*

Problème 4

Données : *Un échiquier avec une partie entamée et $n \in \mathbb{N}$.*
Question : *Est-ce que les blancs peuvent gagner en n coups ?*

À vous de jouer !

Regardez bien ces quatre problèmes. Lequel vous semble le plus “facile” ? Lequel vous semble le plus “difficile” ? Gardez votre réponse en tête => on y reviendra.

On dit qu’une **instance** d’un problème est composée d’une valeur concrète pour chaque donnée du problème. C’est la version “avec des vrais chiffres dedans”.

Exemple 1.2
“30” est une instance du problème 3, et la réponse est oui. $(5 \times 3 + 4)$ *est une autre instance (réponse : non).*

1.1.1 Problèmes de décision et problèmes d’optimisation

Dans ce cours, nous nous intéresserons principalement aux **problèmes de décision** : des problèmes dont la réponse est **oui** ou **non**.

À retenir

Un **problème de décision** est un problème dont la réponse est toujours soit "oui", soit "non".

Exemple 1.3

- *"n est-il divisible par 10 ?"* → *problème de décision (réponse : oui ou non).*
- *"Quel est le reste de la division de n par 10 ?"* → pas *un problème de décision (la réponse est un nombre).*
- *"Quel est le plus court chemin entre A et B ?"* → *problème d'**optimisation**. Mais on peut le transformer en problème de décision : "Existe-t-il un chemin de longueur $\leq k$ entre A et B ?"*

À vous de jouer !

Pourquoi se concentrer sur les problèmes de décision ? Parce que si on ne sait même pas répondre "oui" ou "non" à une question, on ne pourra certainement pas calculer une réponse complète. Les problèmes de décision sont le cas le plus simple et pourtant, comme on va le voir, même certains d'entre eux sont **impossibles** à résoudre !

1.2 Algorithme

Maintenant qu'on sait ce qu'est un problème, l'étape suivante consiste à essayer de le résoudre. Et pour ça, on a besoin d'un **algorithme**.

À retenir

Un **algorithme** est une méthode indiquant **sans ambiguïté** des actions **mécaniques** à effectuer pour trouver la réponse à un problème.

Deux mots-clés importants dans cette définition :

- **Sans ambiguïté** : tout le monde comprend la méthode de la même façon. "Saler à votre convenance" n'est *pas* sans ambiguïté; votre convenance n'est pas la mienne.
- **Mécanique** : il ne fait pas appel à l'intelligence ni à la réflexion. Un robot pourrait l'exécuter. D'ailleurs, c'est exactement ce que fait un ordinateur : c'est un robot qui exécute des algorithmes.

Exemple 1.4

Voici un algorithme pour le problème 1 (la pâte à crêpes) :

1. *Mettre la farine dans une terrine.*
2. *Faire un puits.*

3. *Y casser les œufs entiers.*
4. *Ajouter l'huile, le sel et 10 cl de lait.*
5. *Travailler énergiquement la pâte avec une cuillère.*
6. *Mouiller progressivement avec le reste du lait.*
7. *Laisser reposer 1 heure.*

Chaque étape est claire, non ambiguë, et mécanique. Même un robot pourrait faire des crêpes.

Exemple 1.5
Voici un algorithme pour le problème 3 (divisibilité par 10) :

1. *Déterminer le reste r de la division de n par 10.*
2. *Si $r = 0$, alors n est divisible par 10. Sinon, il ne l'est pas.*

Deux étapes. Simple. Efficace.

Un **programme**, c'est la traduction d'un algorithme dans un langage de programmation. C'est le passage de la théorie à la pratique.

Exemple 1.6
Voici un programme en C pour le problème 3 :

```
#include <stdio.h>
int main(){
  int n, r;
  printf("Entrez votre nombre : ");
  scanf("%d", &n);
  r = n % 10;
  if (r == 0){
    printf("%d est divisible par 10.\n", n);
  } else {
    printf("%d n'est pas divisible par 10.\n", n);
  }
  return 0;
}
```

Quiz rapide

Parmi les instructions suivantes, laquelle n'est **pas** compatible avec la définition d'un algorithme ?

a) "Ajouter 3 grammes de sel."

b) "Saler selon votre goût."

c) "Diviser n par 2."

d) “Si le résultat est pair, écrire OUI.”

Réponse

b) “Saler selon votre goût” est ambigu.

1.3 Existe-t-il un algorithme pour chaque problème ?

C’est LA question fondamentale de ce cours. Et la réponse va peut-être vous surprendre.

À vous de jouer !

Avant de lire la suite, prenez 30 secondes pour y réfléchir. Selon vous, peut-on trouver un algorithme pour *n’importe quel* problème ?

La réponse est **non**.

Oui, il existe des problèmes pour lesquels **aucun** algorithme ne peut donner la réponse. Pas parce qu’on n’est pas assez malins pour le trouver, pas parce qu’on n’a pas assez de puissance de calcul, mais parce que **c’est mathématiquement impossible**. Aucun algorithme, aucun programme, aucun ordinateur aussi puissant soit-il ne pourra jamais résoudre ces problèmes.

1.3.1 Un exemple concret : le problème de pavage

Imaginez un jeu de tuiles carrées. Chaque côté d’une tuile est d’une certaine couleur. Les règles sont simples : vous devez recouvrir tout le plan infini avec ces tuiles, sans les faire pivoter, en les plaçant côte à côte de sorte que les couleurs des côtés qui se touchent soient identiques.

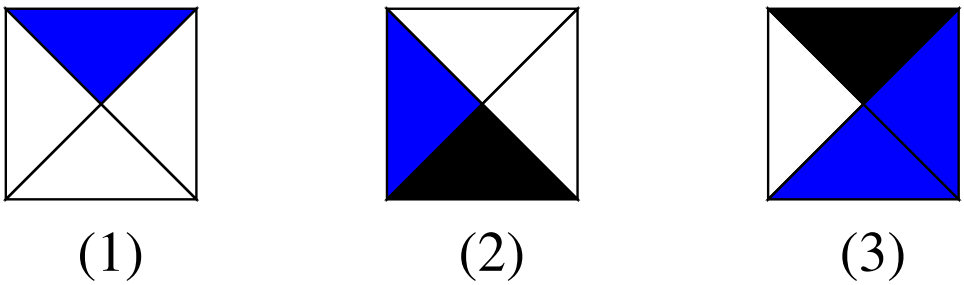


Figure 1.1: Une instance du problème de pavage pour laquelle une solution existe.

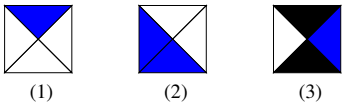


Figure 1.2: Une instance du problème de pavage pour laquelle aucune solution n’existe.

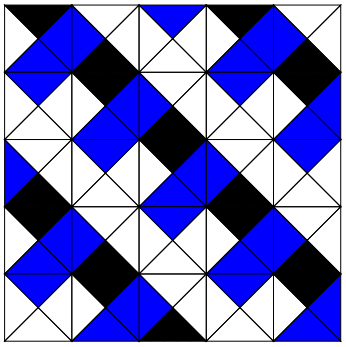


Figure 1.3: Solution pour la première instance.

La question que l'on se pose est la suivante : pour un ensemble donné de tuiles, est-il possible de paver le plan ?

Au début des années 1960, le logicien Hao Wang étudie ce problème. Il observe que tous les exemples qu'il connaît de pavages réussis ont une propriété particulière : ils sont périodiques. Cela signifie qu'à partir d'un certain point, le motif de tuiles se répète identiquement à l'infini, comme un papier peint. Wang formule alors une conjecture, connue sous le nom de conjecture de Wang :

Si un ensemble fini de tuiles permet de paver le plan, alors il en existe forcément un pavage périodique.

Si cette conjecture était vraie, le problème de savoir si un ensemble pave le plan deviendrait potentiellement plus simple. Il suffirait de chercher un motif de base qui se répète.

En 1966, Robert Berger, un doctorant de Wang, démontre que la conjecture de son directeur de thèse est fausse. Pour cela, il construit un ensemble colossal de 20 426 tuiles qui possède une propriété extraordinaire :

Il pave le plan : on peut disposer ces tuiles à l'infini en respectant toutes les règles.

Mais il le fait uniquement de manière apériodique : le motif obtenu ne se répète jamais. Il n'y a pas de "motif de base" qui se répète. Le pavage est là, il est infini, mais il est unique et sans cesse changeant.

Mais Berger ne s'arrête pas là. Il va utiliser cette propriété d'apériodicité pour établir un résultat bien plus profond et fondamental. Il montre que le problème du pavage de Wang est indécidable. Autrement dit, il n'existe et il n'existera aucun algorithme universel capable de répondre correctement à la question "cet ensemble de tuiles pave-t-il le plan ?" pour tous les ensembles de tuiles possibles.

1.3.2 Que signifie “aucun algorithme n'existe” ?

Soyons précis, car c'est un point subtil.

⚠ À retenir

Dire qu'aucun algorithme n'existe pour un problème, cela signifie qu'il n'existe pas d'algorithme qui réponde correctement à la question pour **n'importe quelle instance** du problème.
Attention à la nuance ! On ne dit *pas* qu'on ne peut résoudre *aucune* instance du problème. On peut très bien résoudre certaines instances. Ce qu'on dit, c'est qu'il

n'existe pas de méthode *universelle* qui marche pour *toutes* les instances.

1.3.3 Décidable, indécidable

À retenir

- Un problème pour lequel **un algorithme existe** et donne toujours la bonne réponse est dit **décidable**.
- Un problème pour lequel **aucun algorithme n'existe** est dit **indécidable**.

Un peu d'histoire

Dans les années 1930, Alonzo Church avance une thèse fondamentale, appelée **Thèse de Church-Turing** :
Il existe un algorithme répondant à un problème donné si et seulement s'il existe une machine de Turing qui détermine la solution de ce problème.
Autrement dit, les machines de Turing qu'on va découvrir au chapitre suivant capturent *exactement* la notion intuitive d'algorithme. Rien de plus, rien de moins.

L'étude des problèmes décidables et indécidables constitue un domaine entier de l'informatique théorique appelé la **calculabilité**. C'est l'un des sujets principaux de ce cours.

1.4 Tous les algorithmes sont-ils utilisables ?

Admettons qu'on ait trouvé un algorithme pour notre problème. Victoire ? Pas si vite. Un algorithme qui met 490 millions d'années à s'exécuter, est-ce vraiment un algorithme "utile" ?

Prenons un exemple concret : le puzzle des singes.

Problème : puzzle des singes

Données : un ensemble de cartes carrées dont l'orientation est donnée et dont les côtés représentent le bas ou le haut d'un singe.

Question : Peut-on arranger ces cartes pour former un grand carré tel que les moitiés de singes se correspondent ?

Un algorithme naïf pour résoudre ce problème : essayer *toutes les combinaisons* possibles de cartes. Si on en trouve une qui marche, on répond oui. Sinon, non.

Ça marche ? Techniquement, oui. L'algorithme finit toujours par s'arrêter et donne toujours la bonne réponse.

Mais : avec 25 cartes (un carré de 5×5), dans le pire des cas, il faut tester 25! combinaisons. 25! est un nombre à **26 chiffres**. Un ordinateur testant 1 milliard de combinaisons par seconde mettrait **490 millions d'années**.

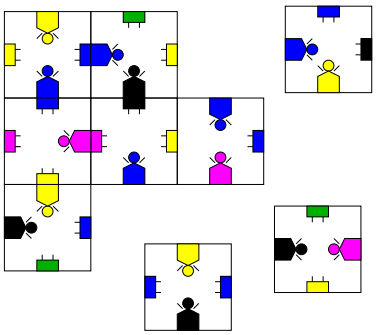


Figure 1.4: Une instance du problème des singes.

1.4.1 Problèmes traitables et non traitables

> **À retenir**
>
> Un problème est dit **traitable** (*tractable* en anglais) s'il existe un algorithme qui le résout en un temps "raisonnable". Sinon, il est **non traitable** (*intractable*).

Les deux principales raisons qui rendent un algorithme inutilisable :

- **Le temps de calcul est trop long.**
- **Il utilise trop de mémoire.**

Ces deux problèmes sont liés : en un temps donné, on ne peut écrire qu'un nombre fini d'informations en mémoire.

Quand un problème est non traitable, tout n'est pas perdu ! On peut chercher :

- des algorithmes qui donnent une **réponse approchée** (pas parfaite, mais acceptable), comme avec des algorithmes gloutons (greedy algorithm)
- des algorithmes qui **ne répondent pas toujours** mais qui, quand ils répondent, donnent la bonne réponse, comme certains algorithmes probabilistes (Las Vegas)

1.5 Complexité et algorithmique

> **À retenir**
>
> La **complexité** est une mesure qui évalue l'efficacité d'un algorithme : son temps d'exécution et la place mémoire utilisée. Elle permet de comparer deux algorithmes qui résolvent le même problème.

L'étude de la complexité des algorithmes est une discipline centrale en informatique. Nous l'aborderons en détail dans les chapitres de ce cours.

L'**algorithmique**, quant à elle, étudie les méthodes pour *améliorer* la complexité d'un algorithme. Nous n'en ferons pas le cœur de ce cours, mais nous en verrons quelques aperçus.

1.6 La grande carte de l'informatique théorique

Voici la "grande carte" que nous allons explorer dans ce cours :

Catégorie	Exemples	Complexité
Indécidables	Problème de l'arrêt, Pavage de Wang	Aucun algorithme n'existe.
Décidables		
$\mathcal{NP}$-complets (probablement non traitables)	SAT, Voyageur de commerce, Sudoku	Temps exponentiel dans le pire cas (sauf si $\mathcal{P} = \mathcal{NP}$)
Traitables ($\mathcal{P}$)	Tri d'une liste, Plus court chemin, Divisibilité	Temps polynomial ("raisonnable")

Et la grande question littéralement à un million de dollars ! : la **frontière entre $\mathcal{P}$ et $\mathcal{NP}$**. Plus précisément : $\mathcal{P}$ regroupe les problèmes qu'on sait **résoudre** efficacement (en temps polynomial), tandis que $\mathcal{NP}$ regroupe les problèmes dont on sait **vérifier** une solution efficacement. La question $\mathcal{P} \stackrel{?}{=} \mathcal{NP}$ demande si tout problème dont on peut vérifier une solution rapidement peut aussi être *résolu* rapidement.

À vous de jouer !

Attention au piège classique : "NP" ne veut **pas** dire "Non Polynomial" ! NP signifie "**N**on-deterministic **P**olynomial". Un problème $\mathcal{NP}$ n'est pas forcément difficile — tous les problèmes de $\mathcal{P}$ sont aussi dans $\mathcal{NP}$ (si vous savez résoudre un problème vite, vous savez aussi vérifier une solution vite). Ce qui est "probablement difficile", ce sont les problèmes **$\mathcal{NP}$-complets** : les plus durs de $\mathcal{NP}$. Nous y reviendrons en détail.

Quiz rapide

Parmi les affirmations suivantes, lesquelles sont vraies ?

a) Tout problème a un algorithme qui le résout.

b) Un problème indécidable est un problème pour lequel aucun algorithme n'existe.

c) Un algorithme qui existe mais qui met des milliards d'années à s'exécuter est quand même un algorithme.

d) Si un problème est non traitable, on ne peut rien faire du tout.

Réponse

b) et **c)** sont vraies. a) est faux (des problèmes indécidables existent). d) est faux (on peut chercher des approximations ou des heuristiques).

Question de réflexion

Votre GPS utilise un algorithme pour calculer le plus court chemin entre deux points. Pensez-vous que ce problème est traitable ? Et si on changeait la question en "trouver le plus court chemin passant par *toutes* les villes d'un pays" (problème du voyageur de commerce), est-ce toujours le cas ? Nous y reviendrons...

La suite... On a vu qu'un algorithme est une méthode mécanique et sans ambiguïté. Mais comment *formaliser* cette notion de manière rigoureuse ? C'est exactement ce que permet la **machine de Turing**, que nous allons découvrir au chapitre suivant.

2 Les machines de Turing

Un peu d'histoire

Nous sommes en 1936. Alan Turing a 24 ans. Il est étudiant à Cambridge et il s'attaque à l'un des problèmes les plus profonds des mathématiques : le *Entscheidungsproblem* (problème de décision) posé par David Hilbert en 1928.
Hilbert croyait qu'on pourrait construire une procédure mécanique capable de déterminer la vérité de n'importe quel énoncé mathématique. Turing va prouver que c'est **impossible**. Pour cela, il invente un modèle de calcul d'une simplicité désarmante : un ruban infini, une tête de lecture, et quelques règles.
Fait remarquable : la même année, indépendamment, l'Américain Emil **Post** invente un modèle quasi identique (les "systèmes de Post"). Et Alonzo **Church** propose une approche entièrement différente (le λ-calcul) qui aboutit aux mêmes conclusions. Trois chemins indépendants vers la même vérité : c'est en soi un argument puissant en faveur de la thèse de Church-Turing.
Turing n'a jamais construit sa machine. C'est une expérience de pensée, un outil mathématique. Mais c'est grâce à elle qu'on a pu définir précisément ce qui est calculable et ce qui ne l'est pas. Et accessoirement, cette idée a mené à l'invention des vrais ordinateurs quelques années plus tard.
(Et si vous vous demandez : oui, c'est le même Turing qui a cassé le code Enigma pendant la Seconde Guerre mondiale.)

À retenir

Objectifs du chapitre :

- Comprendre le modèle formel de la **machine de Turing**.
- Maîtriser les notions de **langage reconnu** et de **langage décidé**.
- Découvrir les variantes : machines à plusieurs rubans, non déterministes.
- Comprendre la **thèse de Church-Turing** et la **machine universelle**.

2.1 Le vocabulaire de base

Avant de plonger dans les machines de Turing, il faut poser quelques définitions.

À retenir

- Un **alphabet** Σ est un ensemble **fini** de symboles. Exemples : $\{a, b\}$, $\{0, 1\}$, $\{A, B, \ldots, Z\}$.
- Un **mot** sur un alphabet Σ est une suite finie de symboles pris dans Σ. Exemples : aba, 001101, $BONJOUR$.
- Le **mot vide**, noté ε, est la suite vide (le mot qui ne contient aucun symbole). C'est le "silence" de l'alphabet.
- Un **langage** est un ensemble de mots. Il peut être fini, infini, ou même vide.
- Σ^* désigne l'ensemble de **tous** les mots qu'on peut former avec l'alphabet Σ (y compris le mot vide).

On définit aussi la notation a^n récursivement :

- $a^0 \stackrel{\text{def}}{=} \varepsilon$ (zéro fois a, c'est le mot vide),
- $a^{n+1} \stackrel{\text{def}}{=} a^n a$ ($n+1$ fois a, c'est n fois a suivi de a).

Exemple 2.1

- $\{a, bab\}$ *est un langage (il contient exactement deux mots).*
- $\emptyset$ *est un langage qui ne contient* **aucun** *mot. Attention, il est distinct de* $\{\varepsilon\}$ *qui contient* **un** *mot (le mot vide). C'est la différence entre "une boîte vide" et "pas de boîte du tout".*
- $\{\omega \in \{a, b\}^* \mid \exists u, v \in \{a, b\}^* \text{ et } \omega = uabv\}$ *est le langage des mots qui contiennent la sous-suite* ab*. Par exemple,* aab, ab, bab, $aabb$ *en font partie.*

Quiz rapide

Soit $\Sigma = \{0, 1\}$. Lesquels de ces éléments appartiennent à Σ^* ?

a) 010

b) ε

c) 2

d) 0000

e) 01201

2.2 Présentation intuitive

Imaginez le dispositif le plus simple possible pour faire du calcul :

1. **Un ruban** : une bande de papier infinie (dans les deux sens), divisée en cases. Chaque case contient un symbole. C'est la *mémoire* de notre machine. La plupart des cases contiennent un symbole spécial B (pour "Blanc" ou "vide").
2. **Une tête de lecture/écriture** : elle est positionnée sur une case du ruban. Elle peut *lire* le symbole qui s'y trouve, *écrire* un nouveau symbole à la place, puis se *déplacer* d'une case vers la gauche ou la droite. C'est le *bus* entre le processeur et la mémoire.
3. **Un ensemble fini d'états** : c'est le "cerveau" de la machine. À chaque instant, la machine est dans un état donné. C'est le *processeur*.
4. **Une table de transitions** : c'est le "programme". Elle dit à la machine : "Si tu es dans tel état et que tu lis tel symbole, alors écris tel symbole, déplace-toi dans telle direction, et passe dans tel état."

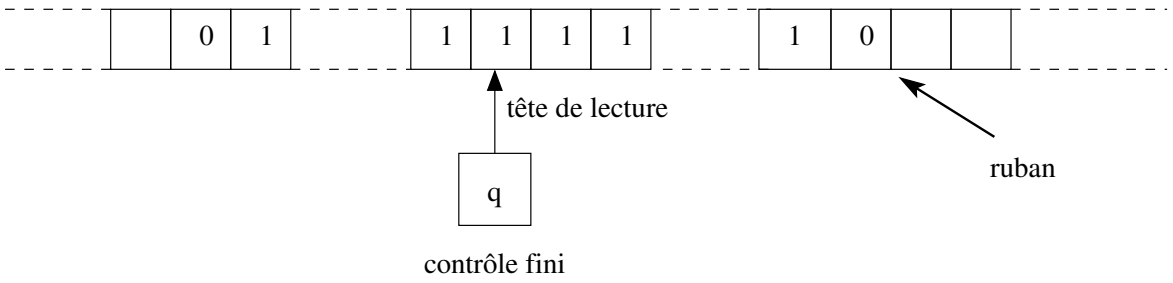


Figure 2.1: Machine de Turing au cours d'un calcul.

2.3 Définition formelle

Une machine de Turing est définie mathématiquement comme un septuplet.

À retenir

Une **machine de Turing** M est un septuplet $(Q, q_0, F, \Gamma, B, \Sigma, \delta)$ où :

- Q est l'ensemble **fini** d'états,
- $q_0 \in Q$ est l'**état initial**,
- $F \subseteq Q$ est l'ensemble des **états finaux** (ou acceptants),
- Γ est l'**alphabet du ruban** (tous les symboles qui peuvent apparaître sur le ruban),
- $B \in \Gamma$ est le symbole **blanc** (case vide),
- $\Sigma \subseteq (\Gamma \setminus \{B\})$ est l'**alphabet d'entrée** (les symboles utilisés pour écrire les données),
- $\delta : Q \times \Gamma \to Q \times \Gamma \times \{G, D\}$ est la **fonction de transition**.

La fonction de transition δ est le cœur de la machine. Elle dit : "Si je suis dans l'état q et que je lis le symbole a, alors j'écris le symbole b, je passe dans l'état q', et je me déplace vers la **G**auche ou la **D**roite."

On note : $\delta(q, a) = (q', b, D)$ signifie "dans l'état q, en lisant a : écrire b, passer en q', aller à droite."

> **À retenir**
>
> **Point important :** la fonction δ est souvent **partielle**, c'est-à-dire qu'elle n'est pas forcément définie pour tous les couples (q, a). Quand $\delta(q, a)$ n'est pas définie, la machine **s'arrête**. C'est le mécanisme fondamental d'arrêt d'une machine de Turing. Par convention, δ n'est jamais définie pour les états finaux $q \in F$, ce qui force la machine à s'arrêter dès qu'elle atteint un état acceptant.

Exemple 2.2
Voici une machine de Turing concrète : $M = (Q, q_0, \{q_f\}, \{1, B\}, B, \{1\}, \delta)$ avec $Q = \{q_0, q_i, q_p, q_f\}$ et δ définie par :

δ	1	B
q_0	$(q_i, 1, D)$	(q_f, B, G)
q_i	$(q_p, 1, D)$	
q_p	$(q_i, 1, D)$	(q_f, B, G)

Quelques observations :

- *L'alphabet d'entrée est $\{1\}$: les données sont des suites de 1.*
- *La case (δ, q_i, B) est vide : la fonction de transition n'est **pas définie** pour q_i lisant B. La machine s'arrête dans ce cas.*
- *q_f est le seul état final.*

Mais que fait *cette machine ? Patience, on va le découvrir en regardant comment elle calcule.*

2.4 Configurations et calculs

2.4.1 Configuration

Pour décrire précisément ce que fait une machine de Turing à un instant donné, on utilise la notion de **configuration**.

> **À retenir**
>
> Une **configuration** d'une machine de Turing capture trois informations :
>
> 1. L'**état** dans lequel se trouve la machine.
> 2. Le **contenu** du ruban.
> 3. La **position** de la tête de lecture.
>
> On la note $\alpha q \beta$ où :

- α est le contenu du ruban à **gauche** de la tête,
- q est l'état courant,
- β est le contenu du ruban à partir de la tête (le premier symbole de β est celui sous la tête).

La **configuration initiale** est $q_0\beta$ où β est le mot donné en entrée. Au début, la tête est sur le premier symbole de l'entrée, et la machine est dans l'état initial q_0.

2.4.2 Étape de calcul

Une étape de calcul consiste à passer d'une configuration à une autre en appliquant la fonction de transition δ.

Concrètement, à chaque étape, la machine :

1. **Change** d'état,
2. **Écrit** un symbole à la place de celui qu'elle vient de lire,
3. **Déplace** sa tête d'une case vers la gauche (G) ou la droite (D).

On note $c \vdash_M c'$ pour dire "la configuration c passe en une étape à la configuration c'". Et $c \vdash_M^* c'$ pour "c aboutit à c' après zéro ou plusieurs étapes".

Formellement :

- Si $\delta(q, a) = (q', b, D)$: $\alpha q a \beta \vdash_M \alpha b q' \beta$ (on écrit b et on va à droite).
- Si $\delta(q, a) = (q', b, G)$: $\alpha' c\, q\, a \beta \vdash_M \alpha' q' c\, b\, \beta$ (on écrit b et on va à gauche).

2.4.3 Exemple de calcul complet

Reprenons la machine de l'exemple 2.2.

2.4.3.0.1 Calcul sur le mot 111 (trois 1) :

$$q_0\,111 \;\vdash\; 1\,q_i\,11 \;\vdash\; 11\,q_p\,1 \;\vdash\; 111\,q_i$$

La machine s'arrête ! Pourquoi ? Parce que q_i lit B (le blanc après les trois 1) et $\delta(q_i, B)$ n'est pas défini.

Résultat : la machine est dans l'état q_i, qui n'est **pas** un état final. Le mot 111 n'est donc **pas accepté**.

2.4.3.0.2 Calcul sur le mot 1111 (quatre 1) :

$$q_0\,1111 \;\vdash\; 1\,q_i\,111 \;\vdash\; 11\,q_p\,11 \;\vdash\; 111\,q_i\,1 \;\vdash\; 1111\,q_p \;\vdash\; 111\,q_f\,1$$

La machine s'arrête dans l'état q_f, qui **est** un état final. Le mot 1111 est donc **accepté**.

À vous de jouer !

Voyez-vous le motif ? La machine alterne entre les états q_i (impair) et q_p (pair) à chaque 1 qu'elle lit. Quand elle arrive au bout du mot, si elle est en q_p (pair), elle passe en q_f et accepte. Sinon, elle se bloque.
Conclusion : cette machine accepte les mots contenant un **nombre pair** de 1. Autrement dit, elle reconnaît le langage des mots de longueur paire sur $\{1\}$.

Quiz rapide

Modifiez cette machine pour accepter uniquement les mots de taille impaire.

2.5 Mots et langages reconnus

Quand une machine de Turing calcule sur un mot, trois choses peuvent se produire :

À retenir

Étant donné un mot $\omega \in \Sigma^*$:

1. Le calcul **s'arrête dans un état final** $\Rightarrow$ ω est **accepté** (reconnu) par M.
2. Le calcul **s'arrête dans un état non final** $\Rightarrow$ ω est **rejeté** par M.
3. Le calcul **ne s'arrête jamais** $\Rightarrow$ la machine **boucle** sur ω (et on attend indéfiniment...).

Le **langage reconnu** par M, noté $L(M)$, est l'ensemble de tous les mots acceptés par M :

$$L(M) = \{\omega \in \Sigma^* \mid q_0\,\omega \vdash_M^* \alpha\,q\,\beta \text{ avec } q \in F\}$$

Pause café

Pensez à la machine de Turing comme un videur de boîte de nuit. Elle examine chaque client (mot) qui se présente. Soit elle dit "Entrez !" (accepté), soit elle dit "Dehors !" (rejeté), soit elle reste indéfiniment à regarder le client sans rien dire (boucle infinie). Le langage reconnu, c'est l'ensemble des clients qui entrent.

2.6 Langages et fonctions décidables

À retenir

Un langage $L \subseteq \Sigma^*$ est **décidable** s'il existe une machine de Turing M telle que :

- $L = L(M)$ (la machine reconnaît exactement L), **et**
- M **s'arrête sur tous les mots** de Σ^*.

Autrement dit : la machine donne **toujours** une réponse (oui ou non), elle ne boucle **jamais**.

À vous de jouer !

La nuance est importante ! "Reconnu" et "décidable", ce n'est pas la même chose. Un langage peut être reconnu par une machine qui boucle sur certains mots (ceux qu'elle n'accepte pas). Un langage décidable exige que la machine réponde *à tout le monde*, même par "non".

Le langage des mots de longueur paire sur $\{1\}$ est décidable : notre machine de l'exemple 2.2 s'arrête sur *tous* les mots et donne la bonne réponse.

2.6.1 Fonctions décidables

On peut aussi utiliser les machines de Turing pour *calculer des fonctions*, pas seulement pour répondre oui/non.

À retenir

Soit $f : \Sigma^* \to \Sigma^*$. Une machine de Turing M **décide** f si, pour tout mot ω :

- M accepte ω si $f(\omega)$ est définie,
- M rejette ω si $f(\omega)$ n'est pas définie,
- Quand $f(\omega)$ est définie, le ruban contient $f(\omega)$ à la fin du calcul.

Exemple 2.3
Voici une machine qui ajoute 1 à un nombre binaire : $M = (Q, q_0, \{q_f\}, \{0, 1, B\}, B, \{0, 1\}, \delta)$ *avec* $Q = \{q_0, q_r, q_f\}$ *et :*

δ	0	1	B
q_0	$(q_0, 0, D)$	$(q_0, 1, D)$	(q_r, B, G)
q_r	$(q_f, 1, G)$	$(q_r, 0, G)$	$(q_f, 1, D)$

Comment ça marche ?

- *État q_0 : on avance vers la droite jusqu'au bout du nombre.*

- *État q_r : on revient vers la gauche en propageant la retenue. Un 1 devient 0 (retenue), un 0 devient 1 (fin de retenue, on s'arrête).*

Calcul sur 101 *(5 en binaire) :*

$$q_0\,101 \vdash 1\,q_0\,01 \vdash 10\,q_0\,1 \vdash 101\,q_0 \vdash 10\,q_r\,1 \vdash 1\,q_r\,00 \vdash q_f\,1\,10$$

Résultat sur le ruban : 110*, qui est bien 6 en binaire.* $5+1=6$*. Ça marche !*
Calcul sur 11 *(3 en binaire) :*

$$q_0\,11 \vdash 1\,q_0\,1 \vdash 11\,q_0 \vdash 1\,q_r\,1 \vdash q_r\,10 \vdash 1\,q_f\,00$$

Résultat : 100*, soit 4 en binaire.* $3+1=4$.

2.7 Variations sur la machine de Turing

On peut se demander : "Et si on ajoutait un deuxième ruban ? Ou plusieurs têtes de lecture ? La machine serait-elle plus puissante ?"

Réponse surprenante : **non**.

2.7.1 Machine à plusieurs rubans

On peut définir des machines avec k rubans et k têtes de lecture. La fonction de transition lit alors k symboles simultanément et déplace chaque tête indépendamment.

Résultat : toute machine à plusieurs rubans peut être simulée par une machine à un seul ruban. Elle sera plus lente (au plus quadratiquement), mais elle pourra faire exactement la même chose.

2.7.2 Machine non déterministe

Dans une machine **déterministe**, pour chaque état q et symbole a, il y a **au plus une** transition possible.

Dans une machine **non déterministe**, il peut y en avoir **plusieurs**. La machine "choisit" laquelle appliquer. On peut imaginer qu'elle explore *tous* les chemins en parallèle.

À retenir

Une machine non déterministe **accepte** un mot s'il existe **au moins un** chemin de calcul qui mène à un état final.
Résultat : toute machine non déterministe peut être simulée par une machine déterministe (mais avec un potentiel ralentissement **exponentiel**).
Pourquoi exponentiel ? L'ensemble des calculs possibles d'une machine non déterministe sur une entrée ω forme un **arbre** :

- La racine est la configuration initiale.
- Chaque nœud a autant de fils qu'il y a de transitions possibles (au plus b, le **facteur de branchement**).
- Chaque branche représente un chemin de calcul.

Si le calcul dure au plus t étapes, l'arbre a une profondeur t et peut contenir jusqu'à b^t feuilles (chemins). La machine déterministe doit explorer **tout l'arbre** (par exemple en parcours en largeur) pour vérifier s'il existe au moins un chemin acceptant. D'où le temps $O(b^t)$, qui est **exponentiel** en t.

À vous de jouer !

Cette différence entre déterministe et non déterministe va devenir *cruciale* quand on parlera de $\mathcal{P}$ vs $\mathcal{NP}$. Retenez bien cette distinction !

2.7.3 La thèse de Church-Turing

Et c'est là que ça devient vraiment impressionnant.

À retenir

Thèse de Church-Turing : tout ce qui est "calculable" au sens intuitif du terme peut être calculé par une machine de Turing.
Autrement dit : peu importe le langage de programmation (Python, Java, C, Haskell...), peu importe l'ordinateur (votre laptop, un superordinateur, un hypothétique ordinateur quantique), tout ce qu'ils peuvent calculer, une machine de Turing peut aussi le calculer.
La réciproque est vraie aussi : ce qu'une machine de Turing ne peut **pas** calculer, **aucun** ordinateur ne le pourra jamais.

Pause café

Cela signifie que votre vieux smartphone des années 2000 et le plus puissant supercalculateur du monde peuvent résoudre *exactement les mêmes problèmes*. La différence n'est qu'une question de **vitesse** et de **mémoire**, pas de **capacité**.

2.8 Machines de Turing comme énumérateurs

Jusqu'ici, on a vu les machines de Turing comme des "décideurs" (qui répondent oui/non) ou des "calculateurs" (qui calculent une fonction). Mais il existe un troisième rôle, tout aussi important.

À retenir

Un **énumérateur** est une machine de Turing qui, au lieu d'accepter ou rejeter un mot, **génère** (imprime) une liste de mots, un par un, sur un ruban de sortie. Elle peut ne jamais s'arrêter (si la liste est infinie), mais chaque mot de la liste est produit en un temps fini.

Un langage L est **récursivement énumérable** (ou **semi-décidable**) s'il existe un énumérateur qui produit exactement les mots de L (éventuellement dans le désordre et avec des répétitions).

Pause café

Pensez à un énumérateur comme un distributeur de billets de loterie. Il imprime des billets un par un. Certains sont gagnants (les mots du langage). Si vous attendez assez longtemps, vous verrez apparaître tous les billets gagnants. Mais vous ne saurez jamais quand il a *fini* d'en produire, peut-être qu'un autre billet gagnant arrivera dans 5 minutes, peut-être jamais.

Résultat fondamental : Un langage L est semi-décidable si et seulement s'il existe une machine de Turing qui **accepte** tous les mots de L (mais peut boucler sur les mots hors de L).

Conséquence : Un langage est **décidable** si et seulement si lui-même *et* son complémentaire sont semi-décidables.

La semi-décidabilité est une notion "à moitié" : la machine peut dire "oui" de manière fiable, mais elle est incapable de dire "non" de manière fiable. Elle reste silencieuse (boucle indéfiniment) quand la réponse est non. On verra au chapitre suivant que le problème de l'arrêt est semi-décidable mais *pas* décidable.

2.9 La machine de Turing universelle

Jusqu'ici, chaque machine de Turing était conçue pour une tâche spécifique : reconnaître des mots de longueur paire, vérifier si un nombre est premier, trier une liste... Mais voici une nouvelle idée : peut-on construire une **seule** machine de Turing qui **simule toutes les autres** ?

Un peu d'histoire

C'est la question qu'Alan Turing pose dans son article fondateur de 1936. John von Neumann lira cet article dans les années 1940 et en tirera les principes de l'**architecture de Von Neumann**, qui structure encore tous les ordinateurs modernes : un processeur qui lit et exécute des instructions stockées en mémoire, traitées *comme des données*. Votre ordinateur, votre téléphone, les serveurs du cloud ;tous sont des réalisations de cette idée.

2.9.1 L'idée clé : un programme est une donnée

Toute machine de Turing M peut être décrite de manière finie : on liste ses états, son alphabet, et sa fonction de transition. Cette description peut elle-même être **codée comme un mot binaire** $\langle M \rangle \in \{0,1\}^*$. L'idée est simple : on numérote les états, les symboles, et on encode chaque transition $\delta(q_i, a_j) = (q_k, a_l, D)$ comme un tuple de nombres. L'ensemble forme un mot fini qui décrit *complètement* la machine.

On verra le codage formel en détail au chapitre suivant (section 3.3.1), mais l'idée essentielle est celle-ci : **un programme n'est rien d'autre qu'une donnée**. Une

machine de Turing peut donc prendre en entrée la *description* d'une autre machine de Turing.

À vous de jouer !

C'est exactement ce qui se passe quand vous lancez Python sur votre ordinateur. Python est un programme (une description de machine) qui lit d'autres programmes (vos scripts `.py`) et les exécute. Votre ordinateur physique est la machine universelle ; Python est la machine simulée ; votre script est l'entrée ω.

2.9.2 Définition formelle

À retenir

Une **machine de Turing universelle** $\mathcal{U}$ est une machine de Turing telle que, pour toute machine de Turing M et tout mot $\omega \in \Sigma^*$:

$$\mathcal{U}(\langle M \rangle, \omega) \text{ simule } M(\omega)$$

Plus précisément :

- Si M **accepte** ω, alors $\mathcal{U}(\langle M \rangle, \omega)$ accepte.
- Si M **rejette** ω, alors $\mathcal{U}(\langle M \rangle, \omega)$ rejette.
- Si M **boucle** sur ω, alors $\mathcal{U}(\langle M \rangle, \omega)$ boucle.

Idée de construction : $\mathcal{U}$ utilise trois rubans :

1. **Ruban de travail** : contient ω, simulant le ruban de M (mis à jour à chaque étape de la simulation).
2. **Ruban de code** : contient $\langle M \rangle$, la description de M (inchangée pendant la simulation).
3. **Ruban d'état** : contient l'état courant de M en cours de simulation.

À chaque pas, $\mathcal{U}$ lit l'état courant (ruban 3) et le symbole sous la tête (ruban 1), parcourt le code de M (ruban 2) pour retrouver la transition $\delta(q, a)$, puis met à jour les trois rubans en conséquence.

À retenir

Théorème : La machine de Turing universelle existe.
Si M s'exécute en temps $T(n)$ sur une entrée de taille n, alors $\mathcal{U}$ la simule en temps $O(T(n) \cdot |\langle M \rangle|)$, un surcoût au plus linéaire en la taille du code de M.

2.9.3 Importance conceptuelle

L'existence de $\mathcal{U}$ a des conséquences profondes :

- **Logiciels :** un ordinateur réel est une approximation de $\mathcal{U}$. Le matériel est fixe, mais le *logiciel* peut lui faire exécuter n'importe quel algorithme.
- **Interpréteurs et compilateurs :** Python, la JVM (Java), le moteur JavaScript de votre navigateur sont des instances de machines universelles.
- **Ordinateurs à programme enregistré :** L'architecture de Von Neumann (CPU + mémoire + programme stocké) est la réalisation physique de $\mathcal{U}$.
- **Limites fondamentales :** l'existence de $\mathcal{U}$ entraîne directement les résultats d'indécidabilité. Puisqu'une machine peut simuler n'importe quelle autre, certaines questions sur le comportement des machines deviennent *fondamentalement* insolubles (voir le chapitre suivant).

Pause café

La machine universelle, c'est comme une console de jeu : au lieu de construire un boîtier différent pour chaque jeu, on construit **une seule machine** capable de lire une cartouche (le programme) et d'exécuter le jeu correspondant. Votre PC, votre smartphone, votre console de jeu *sont* tous des machines universelles avec une mémoire finie en pratique, mais l'idée est la même.
La différence avec les machines spécialisées ? En changeant le *logiciel*, on change le comportement sans toucher au *matériel* (hardware/software).

Question de réflexion

Si $\mathcal{U}$ peut simuler toute machine de Turing, peut-elle se simuler elle-même ? Que se passe-t-il si on lui donne son propre code $\langle \mathcal{U} \rangle$ en entrée, et qu'on lui pose la question "vas-tu t'arrêter ?"

2.10 La hiérarchie de Chomsky

Les machines de Turing ne sont pas les seuls modèles de calcul. Les informaticiens et linguistes ont identifié une **hiérarchie** de classes de langages, chacune associée à un type d'automate de puissance croissante. Cette classification est due à **Noam Chomsky** (1956), initialement développée pour modéliser la grammaire des langues naturelles, puis adoptée par l'informatique théorique.

Un peu d'histoire

Noam Chomsky est surtout connu comme linguiste et intellectuel politique. Mais ses travaux sur les grammaires formelles ont profondément influencé la conception des compilateurs. Chaque fois que votre compilateur vérifie que votre code est syntaxiquement correct, il utilise la théorie des grammaires hors-contexte de Chomsky.

Le terme *parsing* (analyse syntaxique) vient directement de cette théorie.

À retenir

La **hiérarchie de Chomsky** classe les langages formels en quatre niveaux imbriqués :

Classe	Grammaire	Reconnaisseur
Langages réguliers	Régulière (Type 3)	Automate fini (AF)
Langages algébriques	Hors-contexte (Type 2)	Automate à pile (AP)
Langages contextuels	Contextuelle (Type 1)	Automate linéairement borné
Langages récursivement énumérables	Sans restriction (Type 0)	Machine de Turing

Les inclusions sont **strictes** :

$$\text{Réguliers} \subsetneq \text{Algébriques} \subsetneq \text{Contextuels} \subsetneq \text{RE}$$

2.10.1 Langages réguliers (Type 3)

Les langages les plus simples. Décrits par des **expressions régulières** (`regex`) et reconnus par des **automates finis** (AF), avec un ensemble fini d'états et **sans mémoire auxiliaire**.

Exemple 2.4

- *Les mots contenant un nombre pair de* `a`.
- *Les identifiants Python :* `[a-zA-Z_][a-zA-Z0-9_]*`.
- *...*

Le langage $\{a^n b^n \mid n \geq 0\}$ *(autant de* a *que de* b*)* ***n'est pas*** *régulier : un automate fini n'a pas de mémoire pour "compter" jusqu'à* n*.*

2.10.2 Langages algébriques / hors-contexte (Type 2)

Reconnus par des **automates à pile** (AP). Ce sont les langages qui décrivent la **syntaxe** des langages de programmation modernes.

Exemple 2.5

- $\{a^n b^n \mid n \geq 0\}$ *: la pile sert à compter les* a*, puis à les débiter en lisant les* b*.*
- *La syntaxe de Python, C, Java : les analyseurs syntaxiques (parsers) des compilateurs exploitent les grammaires hors-contexte.*
- *Les expressions arithmétiques bien parenthésées :* $(1 + (2 \times 3))$.

$\{a^n b^n c^n \mid n \geq 0\}$ ***n'est pas*** *algébrique : une seule pile ne suffit pas à vérifier deux compteurs simultanément.*

2.10.3 Langages contextuels (Type 1)

Reconnus par des **automates linéairement bornés** (ALB) Comme des machines de Turing dont le ruban est limité à la taille de l'entrée.

Exemple 2.6
$\{a^n b^n c^n \mid n \geq 0\}$ *est contextuel (mais pas algébrique).*

2.10.4 Les machines de Turing

Les **machines de Turing** (sans restriction) reconnaissent la classe la plus grande : les langages **récursivement énumérables** (RE). Elles sont au sommet de la hiérarchie.

À retenir

À l'intérieur des langages RE, on distingue encore :

- Les langages **décidables** (R) : la machine s'arrête toujours et donne la bonne réponse. C'est le sous-ensemble des problèmes que les algorithmes peuvent vraiment *résoudre*.
- Les langages **semi-décidables** (RE \ R) : la machine accepte les mots du langage mais peut boucler sur les autres (voir section 2.8).

Pause café

Imaginez des poupées russes.
En pratique :

- Les `regex` de votre éditeur → Type 3 (réguliers).
- La syntaxe de Python ou de C → Type 2 (hors-contexte).
- Les algorithmes que vous écrivez → Type 0 (décidables).
- Ce qu'aucun algorithme ne peut décider → au-delà du sommet.

2.11 Récapitulatif

Quiz rapide

Vrai ou faux ?

a) Une machine de Turing a une mémoire finie.

b) Si $\delta(q, a)$ n'est pas définie, la machine s'arrête.

c) Une machine de Turing non déterministe est plus puissante qu'une machine déterministe (elle reconnaît plus de langages).

d) Le langage reconnu par une machine de Turing est toujours décidable.

e) La thèse de Church-Turing affirme que Python est le meilleur langage de programmation.

Réponse

a) **Faux.** Le ruban est infini. C'est l'ensemble des *états* qui est fini.

b) **Vrai.** La machine ne sait pas quoi faire, elle s'arrête.

c) **Faux.** Elles reconnaissent les *mêmes* langages. La machine non déterministe peut juste être plus rapide (ou pas, c'est justement la question P vs NP !).

d) **Faux.** Un langage reconnu n'est pas forcément décidable. La machine pourrait boucler sur certains mots non reconnus. Pour être décidable, il faut que la machine s'arrête sur *tous* les mots.

e) **Faux.** La thèse dit que tout ce qui est calculable peut l'être par une machine de Turing, quel que soit le langage.

Question de réflexion

On a dit qu'une machine non déterministe peut être simulée par une machine déterministe, mais potentiellement avec un ralentissement exponentiel. Pensez-vous que ce ralentissement est *inévitable* ? Ou bien existe-t-il peut-être une simulation plus astucieuse qui ne ralentisse que polynomialement ? (Si vous trouvez la réponse à cette question, vous gagnerez un million de dollars.)

La suite... Nous avons maintenant un modèle de calcul universel. Mais que se passe-t-il quand on demande à une machine de Turing d'analyser une *autre* machine de Turing ? C'est là que les choses deviennent intéressantes : au chapitre suivant, nous découvrirons les **limites fondamentales du calcul** et le célèbre **problème de l'arrêt**.

3 Décidabilité et indécidabilité

À retenir

Objectifs du chapitre :

- Comprendre la différence entre **décidable** et **semi-décidable**.
- Découvrir des problèmes **indécidables** : l'acceptation, l'arrêt.
- Maîtriser les techniques de **réduction** pour prouver l'indécidabilité.
- Comprendre le **théorème de Rice** et ses conséquences pratiques.

3.1 Rappel : la thèse de Church-Turing

Avant de plonger dans l'indécidabilité, rappelons le pont entre "algorithme" et "machine de Turing".

À retenir

Thèse de Church-Turing (rappel) : tout ce qui est "calculable" au sens intuitif du terme peut être calculé par une machine de Turing (voir chapitre 2).
Conséquence pratique : tout langage de programmation non trivial est équivalent à une machine de Turing. Il existe donc un algorithme pour résoudre un problème si et seulement s'il existe une machine de Turing qui le décide.

Cela signifie que pour prouver qu'un problème est indécidable, il *suffit* de prouver qu'aucune machine de Turing ne peut le décider. Ce résultat s'applique à **tout** ordinateur, **tout** langage de programmation, **tout** système de calcul imaginable (y compris ceux qui n'existent pas encore).

Autrement dit, la différence entre Java, Python, C++, ou n'importe quel autre langage n'est **pas** dans ce qu'ils permettent de programmer, mais dans ce qui est **facile** ou **pratique** à programmer avec chacun.

3.2 Langages et problèmes décidables

Rappelons la définition :

À retenir

Le langage $L \subseteq \Sigma^*$ est **décidable** s'il existe une machine de Turing M telle que :

- $L = L(M)$ (M reconnaît exactement les mots de L),
- M **s'arrête** sur tous les mots $\omega \in \Sigma^*$.

Un problème est **décidable** s'il existe une machine de Turing qui s'arrête sur toutes les instances en donnant la bonne réponse.

3.2.1 Propriétés de fermeture des langages décidables

Les langages décidables ont de belles propriétés. Ce sont de "bons élèves" qui se comportent bien.

3.2.1.1 Le complémentaire d'un langage décidable est décidable.

Si on sait dire "oui" ou "non" pour chaque mot d'un langage L, alors on sait aussi dire "non" ou "oui" , il suffit d'inverser la réponse !

Idée de la preuve : Soit M la machine qui décide L. On construit M' qui fait exactement la même chose que M, sauf qu'à la fin :

- Là où M accepte, M' rejette.
- Là où M rejette, M' accepte.

Concrètement : on ajoute un nouvel état q_n (pour "non"), et partout où M s'arrêtait dans un état non final, M' va dans q_n qui est son seul état final. Les anciens états finaux de M ne sont plus finaux dans M'.

Exemple 3.1
Reprenons notre machine qui reconnaît les mots de longueur paire sur $\{1\}$ *:*

δ	1	B
q_0	$(q_i, 1, D)$	(q_f, B, G)
q_i	$(q_p, 1, D)$	
q_p	$(q_i, 1, D)$	(q_f, B, G)

La machine M' *qui reconnaît les mots de longueur* ***impaire*** *:*

δ'	1	B
q_0	$(q_i, 1, D)$	
q_i	$(q_p, 1, D)$	(q_n, B, G)
q_p	$(q_i, 1, D)$	

L'état final est maintenant q_n *(au lieu de* q_f*). On a ajouté une transition pour* q_i *lisant* B *: au lieu de rester bloqué, la machine va dans l'état final* q_n*.*

3.2.1.1.1 L'intersection et l'union de deux langages décidables sont décidables.

- **Intersection** : on fait tourner M_1 sur le mot. Si M_1 accepte, on fait tourner M_2. Le mot est accepté seulement si les *deux* machines acceptent.
- **Union** : on fait tourner M_1 et M_2 "en parallèle" (sur deux copies du mot). Le mot est accepté si *au moins une* machine accepte.

Quiz rapide

Si L_1 et L_2 sont décidables, lesquels de ces langages sont décidables ?

a) $\overline{L_1}$ (le complémentaire)

b) $L_1 \cap L_2$ (l'intersection)

c) $L_1 \cup L_2$ (l'union)

d) Tous les précédents

d) Tous ! Les langages décidables sont fermés par complémentation, intersection et union.

3.3 Problèmes indécidables

Passons aux choses qui fâchent. Il existe des problèmes que **aucune** machine de Turing ne peut décider. Aucun algorithme, aucun programme, quelle que soit sa complexité.

À retenir

Un problème est **indécidable** s'il n'existe pas de machine de Turing qui le décide. C'est-à-dire que pour **toute** machine de Turing M, il existe au moins une instance sur laquelle :

- soit M ne s'arrête pas,
- soit M donne la mauvaise réponse.

3.3.1 Codage des machines de Turing

Imaginons une idée bizarre : **décrire une machine de Turing sous forme de mot**. Autrement dit, une machine de Turing peut être une **donnée** pour une autre machine de Turing !

À vous de jouer !

C'est exactement ce qui se passe pour un compilateur, un débogueur, un antivirus…, le programme est *à la fois* un outil et une donnée.

On peut coder n'importe quelle machine de Turing comme un mot sur $\{0,1\}^*$. On note $\langle M \rangle$ le codage de la machine M.

Exemple 3.2
On associe à chaque symbole et état un nombre de 0 *:*

0	1	B	G	D	q_0	q_1	q_i	q_p
0	00	000	0000	00000	000000	0000000	00000000	000000000

On sépare les symboles par des 1*. Ainsi, la transition* $\delta(q_0, 1) = (q_i, 1, D)$ *se code :*
$1\underbrace{000000}_{q_0}1\underbrace{00}_{1}1\underbrace{00000000}_{q_i}1\underbrace{00}_{1}1\underbrace{00000}_{D}1$
La machine entière est le mot obtenu en concaténant tous les codes de ses transitions.

3.3.2 Le langage diagonal L_d

On va maintenant construire un langage qui est **prouvablement** indécidable; grance à la **diagonalisation** inspirée de la preuve de Cantor sur les infinis.

3.3.2.1 Construction.

1. On énumère tous les mots de $\{0,1\}^*$ dans l'ordre : d'abord par longueur, puis par ordre lexicographique. On appelle ω_i le i-ème mot.
2. À chaque mot ω_j, on associe une machine de Turing M_j : si ω_j est le codage valide d'une machine, c'est cette machine ; sinon, M_j est la machine qui reconnaît le langage vide.
3. On définit le **langage diagonal** :

$$L_d = \{\omega_i \mid \omega_i \notin L(M_i)\}$$

En français : L_d contient le mot ω_i si et seulement si la machine M_i **n'accepte pas** ω_i. C'est le mot qui dit "je ne suis pas accepté par la machine que je représente".

> **À retenir**
>
> **Théorème** : L_d est indécidable.

Preuve (par l'absurde) :
Supposons que L_d soit décidable. Alors il existe une machine de Turing M qui le décide. Comme M est une machine de Turing, elle a un codage, et il existe un indice j_0 tel que $M = M_{j_0}$.
Question fatale : est-ce que $\omega_{j_0} \in L(M_{j_0})$?

- **Cas 1 :** Supposons $\omega_{j_0} \in L(M_{j_0})$. Puisque M_{j_0} décide L_d (hypothèse), on a $L(M_{j_0}) = L_d$, donc $\omega_{j_0} \in L_d$. Or, par *définition* de L_d, $\omega_{j_0} \in L_d$ signifie $\omega_{j_0} \notin L(M_{j_0})$. On obtient simultanément $\omega_{j_0} \in L(M_{j_0})$ et $\omega_{j_0} \notin L(M_{j_0})$. **Contradiction !**
- **Cas 2 :** Supposons $\omega_{j_0} \notin L(M_{j_0})$. Comme $L(M_{j_0}) = L_d$, on a $\omega_{j_0} \notin L_d$. Or, par *définition* de L_d, $\omega_{j_0} \notin L_d$ signifie $\omega_{j_0} \in L(M_{j_0})$. On obtient simultanément $\omega_{j_0} \notin L(M_{j_0})$ et $\omega_{j_0} \in L(M_{j_0})$. **Contradiction !**

Dans les deux cas, on arrive à une contradiction. Donc notre hypothèse de départ est fausse : L_d n'est pas décidable. □

Visualisation par un tableau. Pour rendre l'argument plus concret, construisons un tableau infini. La ligne i correspond au mot ω_i, la colonne j à la machine M_j. On note "A" si M_j accepte ω_i et "R" sinon.

	M_1	M_2	$\cdots$	M_{j_0}	$\cdots$
ω_1	**A**	R	$\cdots$	A	$\cdots$
ω_2	R	**R**	$\cdots$	R	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	
ω_{j_0}	A	R	$\cdots$	**???**	$\cdots$
$\vdots$	$\vdots$	$\vdots$		$\vdots$	$\ddots$

Le langage L_d est construit en **inversant la diagonale** (les cases en gras) : $\omega_i \in L_d$ si et seulement si la case diagonale (i, i) contient "R". Si $M = M_{j_0}$ décidait L_d, la colonne j_0 serait l'inverse de la diagonale. Mais que vaut la case (ω_{j_0}, M_{j_0}) ? Elle devrait être **à la fois** l'inverse d'elle-même et elle-même. **Contradiction !**

Pause café

Cette preuve a la même structure que le fameux paradoxe du barbier : "Dans un village, le barbier rase tous ceux qui ne se rasent pas eux-mêmes. Qui rase le barbier ?"
Si le barbier se rase lui-même, alors il ne devrait pas (car il ne rase que ceux qui ne se rasent pas). S'il ne se rase pas, alors il devrait (car le barbier rase ceux qui ne se rasent pas). C'est la même logique auto-référentielle !

Quiz rapide

La preuve de l'indécidabilité de L_d utilise :

a) Un argument par induction

b) Un argument par l'absurde (contradiction)

c) Un argument par récurrence

d) Un argument probabiliste

b). On suppose que L_d est décidable, et on montre que cela mène à une contradiction. C'est un raisonnement par l'absurde classique, combiné avec la technique de diagonalisation.

3.4 Réductions

On a maintenant un premier problème indécidable (L_d). Comment montrer que d'autres problèmes sont indécidables ? On utilise la technique de la **réduction**.

À retenir

L'idée d'une réduction est : si on pouvait résoudre le problème B, alors on pourrait aussi résoudre le problème A. Or on *sait* que A est indécidable. Donc B est aussi indécidable.
C'est un raisonnement par *contraposition* :

- Si "B décidable" implique "A décidable",
- et si A est indécidable,
- alors B est indécidable.

Pause café

On *réduit* un problème connu comme impossible à notre nouveau problème. Si le nouveau était possible, l'ancien le serait aussi.

À retenir

Définition formelle. Une **réduction** du langage A au langage B est une fonction **calculable** $f : \Sigma^* \to \Sigma^*$ telle que, pour tout mot ω :

$$\omega \in A \Leftrightarrow f(\omega) \in B$$

On note $A \leq B$ ("A se réduit à B").
Propriété fondamentale : si $A \leq B$ et B est décidable, alors A est décidable. Par contraposée : si A est indécidable et $A \leq B$, alors B est indécidable.

À vous de jouer !

Attention au sens de la réduction ! $A \leq B$ signifie que A est **au plus aussi difficile** que B, pas l'inverse. On transforme une instance de A en une instance de B. Si on sait résoudre B, on sait résoudre A (en transformant puis en résolvant). Donc B est *au moins* aussi difficile que A.

3.4.1 Le problème de l'acceptation (problème universel)

Problème universel (ou de l'acceptation)

Données : Une machine de Turing M et un mot $\omega \in \Sigma^*$

Question : Est-ce que M accepte ω ?

À retenir

Théorème : Le problème de l'acceptation est indécidable.

Idée de la preuve : On réduit L_d au complémentaire du problème de l'acceptation. Si on pouvait décider "M n'accepte pas ω", on pourrait décider L_d (en donnant la machine M_i et le mot ω_i). Or L_d est indécidable. Donc le complémentaire du problème de l'acceptation est indécidable. Et comme le complémentaire d'un langage décidable est décidable, le problème de l'acceptation lui-même ne peut pas être décidable.

À vous de jouer !

En d'autres termes : il est **impossible** d'écrire un programme qui prend en entrée *n'importe quel* autre programme et *n'importe quelle* donnée, et qui répond toujours correctement "oui, ce programme accepte cette donnée" ou "non, il ne l'accepte pas". Aucun débogueur, aucune IA, aucun système ne pourra jamais faire ça de manière fiable pour *tous* les programmes.

3.4.2 Le problème de l'arrêt

Problème de l'arrêt

Données : Une machine de Turing M et un mot $\omega \in \Sigma^*$

Question : Est-ce que M s'arrête sur ω ?

À retenir

Théorème : Le problème de l'arrêt est indécidable.

À vous de jouer !

Il est impossible d'écrire un programme qui détermine si un autre programme va s'arrêter ou tourner indéfiniment.

Idée de la preuve : On réduit le problème de l'acceptation au problème de l'arrêt. L'argument est le suivant :

Supposons qu'il existe une machine M_{arr} qui décide le problème de l'arrêt. On construit une machine M' qui décide le problème de l'acceptation :

1. Sur la donnée (M, ω), on demande à M_{arr} si M s'arrête sur ω.
2. Si la réponse est "non" (la machine M ne s'arrête pas), alors M n'accepte certainement pas ω : on rejette.
3. Si la réponse est "oui" (la machine M s'arrête), on simule M sur ω (on sait qu'elle va s'arrêter !) et on accepte si et seulement si M accepte.

M' déciderait le problème de l'acceptation. Or on a prouvé que c'est impossible. Donc M_{arr} ne peut pas exister.

Pause café

Le problème de l'arrêt a des conséquences pratiques. Il explique pourquoi :

- Les compilateurs ne peuvent pas toujours détecter les boucles infinies.
- Les antivirus ne peuvent pas détecter *tous* les virus.
- La vérification automatique de programmes a des limites fondamentales.

Ce n'est pas une question de technologie ou de budget, c'est une **impossibilité mathématique**.
C'est aussi pourquoi votre ordinateur affiche parfois *"Installation de la mise à jour... Ne pas éteindre"* pendant ce qui semble être une éternité. Votre ordinateur n'est peut etre pas planté mais il ne peut pas non plus vous *prouver* que l'installation va aboutir. 😊

Pause café

La conjecture de Collatz.
Prenez un entier positif n. Si n est pair, divisez-le par 2. S'il est impair, calculez $3n + 1$. Recommencez. Par exemple, en partant de 12 :

$$12 \to 6 \to 3 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$$

La **conjecture de Collatz** affirme que cette suite atteint *toujours* 1, quel que soit l'entier de départ. Personne n'a réussi à la prouver (ni à la réfuter) depuis 1937.
Ce qui est fascinant, c'est que John Conway a montré en 1972 qu'une *version généralisée* de ce problème est **indécidable** par réduction depuis le problème de l'arrêt : il n'existe pas d'algorithme capable de déterminer, pour toute variante de ce type de suite, si elle atteint 1 ou non.

3.4.2.1 Preuve directe par auto-référence (diagonalisation)

La preuve ci-dessus réduit le problème de l'arrêt au problème de l'acceptation. Il existe une preuve plus directe qui construit explicitement une **contradiction par diagonalisation**.

À retenir

Preuve directe de l'indécidabilité de l'arrêt :
Hypothèse : supposons qu'il existe une machine M_{HALT} qui décide le problème de l'arrêt :

$$M_{\mathsf{HALT}}(\langle M, \omega \rangle) = \begin{cases} \text{accepte} & \text{si } M \text{ s'arrête sur } \omega \\ \text{rejette} & \text{si } M \text{ boucle sur } \omega \end{cases}$$

Construction de la machine diagonale D : On construit une machine D qui, sur l'entrée $\langle M \rangle$ (le code d'une machine), procède ainsi :

1. Appelle $M_{\mathsf{HALT}}(\langle M, \langle M \rangle \rangle)$ demande si M s'arrête sur *son propre code*.
2. Si la réponse est "s'arrête" : D **boucle indéfiniment**.
3. Si la réponse est "ne s'arrête pas" : D **s'arrête**.

En pseudocode :

```
Machine D(code_M):
    si M_HALT(code_M, code_M) = "s'arrete":
        boucle infinie
    sinon:
        s'arrete
```

La contradiction : Que se passe-t-il quand on exécute $D(\langle D \rangle)$; on donne à D son *propre code* ?

- **Si D s'arrête sur $\langle D \rangle$** : alors M_{HALT} répond "s'arrête", donc D entre en boucle infinie à l'étape 2. **Contradiction** : D s'arrêterait et ne s'arrêterait pas simultanément.
- **Si D boucle sur $\langle D \rangle$** : alors M_{HALT} répond "ne s'arrête pas", donc D s'arrête à l'étape 3. **Contradiction** : D bouclerait et s'arrêterait simultanément.

Dans les deux cas, contradiction. Donc M_{HALT} n'existe pas. □

Quiz rapide

Quel est le rôle de la machine D dans la preuve directe ?

a) Elle prouve que M_{HALT} s'arrête toujours.

b) Elle exhibe une entrée sur laquelle M_{HALT} donnerait une réponse incorrecte quelle que soit la réponse.

c) Elle calcule le langage de l'arrêt.

d) Elle montre que toute machine de Turing boucle sur son propre code.

Réponse : b). La machine D, appliquée à son propre code, crée un cas où la supposée machine M_{HALT} ne peut donner aucune réponse correcte : quoi qu'elle réponde, on aboutit à une contradiction.

3.5 Semi-décidabilité

Entre décidable et complètement indécidable, il existe une zone intermédiaire.

À retenir

Un langage L est **semi-décidable** (ou **récursivement énumérable**, noté **RE**) s'il existe une machine de Turing M telle que :

- Pour tout $\omega \in L$: M **s'arrête et accepte** ω.
- Pour tout $\omega \notin L$: M **ne s'arrête pas** ou **rejette** ω.

Autrement dit, la machine peut dire "oui" de manière fiable, mais elle peut **boucler indéfiniment** sur les mots qui ne sont pas dans L. Elle ne sait pas dire "non" de manière fiable.
Théorème : Un langage L est **décidable** si et seulement si L *et* $\overline{L}$ (son complémentaire) sont tous les deux semi-décidables.

Idée de la preuve : si on a une machine M_1 qui s'arrête quand $\omega \in L$ et une machine M_2 qui s'arrête quand $\omega \notin L$, on les fait tourner "en parallèle" (en alternant une étape de chaque). L'une des deux finira forcément par s'arrêter (car ω est soit dans L, soit dans $\overline{L}$). On a alors une machine qui s'arrête toujours et qui décide L.

Exemple 3.3
*Le **problème de l'arrêt** est **semi-décidable** : on peut simuler la machine M sur ω pas à pas. Si M s'arrête, on répond "oui". Mais si M ne s'arrête pas, notre simulation ne s'arrête pas non plus, on ne saura jamais dire "non".*

*En revanche, son complémentaire ("M ne s'arrête **pas** sur ω") n'est **pas** semi-décidable. Si c'était le cas, le problème de l'arrêt serait décidable (par le théorème ci-dessus), ce qui est une contradiction.*

3.5.1 La classe co-RE et la hiérarchie R / RE / co-RE

On peut maintenant dresser une **carte complète** des classes de langages autour de la décidabilité.

À retenir

- **R** (langages **récursifs**, ou décidables) : il existe une machine de Turing qui s'arrête sur toutes les entrées et donne toujours la bonne réponse.
- **RE** (récursivement énumérables, semi-décidables) : il existe une machine de Turing qui accepte les mots du langage, mais peut boucler sur les mots qui n'en font pas partie.
- **co-RE** : les langages dont le **complémentaire** est dans RE. Autrement dit, $L \in$ co-RE s'il existe une machine qui s'arrête et rejette les mots *hors de* L, mais peut boucler sur les mots *dans* L.

Théorème fondamental :

$$\mathbf{R} = \text{RE} \cap \text{co-RE}$$

999 Un langage est **décidable** si et seulement si lui-même *et* son complémentaire sont tous les deux semi-décidables.

Langage	Classe	Description
Problème de l'acceptation L_{acc}	RE \ R	$\{\langle M, \omega\rangle \mid M \text{ accepte } \omega\}$
$\overline{L_{\text{acc}}}$	co-RE \ R	$\{\langle M, \omega\rangle \mid M$ rejette ou boucle$\}$
Problème de l'arrêt L_{HALT}	RE \ R	$\{\langle M, \omega\rangle \mid M \text{ s'arrête sur } \omega\}$
$\overline{L_{\text{HALT}}}$	co-RE \ R	$\{\langle M, \omega\rangle \mid M \text{ boucle sur } \omega\}$
$\{a^n b^n \mid n \geq 0\}$	R	Langage algébrique, décidable

À vous de jouer !

L'ensemble de *tous* les langages est **bien plus grand** que RE ∪ co-RE. Il existe des langages qui ne sont ni dans RE ni dans co-RE. Par un argument de comptage (il y a une infinité non dénombrable de langages sur $\{0,1\}^*$, mais seulement une infinité dénombrable de machines de Turing), la *quasi-totalité* des langages est totalement hors de portée de toute machine de Turing.

3.6 Le théorème de Rice

Après avoir prouvé l'indécidabilité du problème de l'arrêt et de l'acceptation au cas par cas, on peut se demander : y a-t-il un résultat plus général ?

La réponse est oui, et c'est le **théorème de Rice** (1953).

À retenir

Théorème de Rice : Toute propriété **non triviale** portant sur le langage reconnu par une machine de Turing est **indécidable**.
Plus formellement : soit P une propriété des langages récursivement énumérables (c'est-à-dire, un ensemble de langages RE). Si P n'est pas triviale (c'est-à-dire, il existe au moins un langage RE qui a la propriété et au moins un qui ne l'a pas), alors le problème "étant donné une machine de Turing M, est-ce que $L(M)$ possède la propriété P ?" est indécidable.

Pause café

En langage courant, le théorème de Rice dit : **on ne peut rien décider d'intéressant sur le comportement d'un programme**.
Voici quelques questions auxquelles aucun algorithme ne peut répondre pour tous les programmes :

- "Ce programme accepte-t-il au moins un mot ?" (indécidable)
- "Ce programme accepte-t-il tous les mots ?" (indécidable)
- "Ce programme reconnaît-il un langage fini ?" (indécidable)
- "Ce programme est-il équivalent à ce programme ?" (indécidable)

Attention : le théorème porte sur le *langage reconnu* (le comportement entrée/sortie), pas sur le code source. Des propriétés "syntaxiques" comme "le programme a-t-il 42 lignes ?" ou "le programme contient-il une boucle `while` ?" sont parfaitement décidables il suffit de lire le code !

Idée de la preuve : On réduit le problème de l'acceptation au problème de décision de P. Si on pouvait décider "$L(M)$ a la propriété P", on pourrait construire une machine qui décide si une machine M_0 accepte un mot ω_0 donné, en construisant une machine M' dont le langage dépend de la réponse. C'est essentiellement une réduction astucieuse.

Pause café

Rice, ChatGPT et la modération de code. Peut-on écrire un programme qui détecte automatiquement si un programme génère du code malveillant ? Le théorème de Rice dit que non (en toute généralité). C'est exactement le défi auquel font face les systèmes de modération automatique de code dans des outils comme GitHub Copilot ou ChatGPT : ils utilisent des heuristiques et des analyses partielles, mais aucun d'entre eux ne peut *garantir* qu'un programme donné est inoffensif.
De même, votre antivirus ne peut pas détecter *tous* les virus possibles : c'est une conséquence directe de Rice. Il fonctionne par signatures connues et heuristiques — pas par analyse sémantique exhaustive (qui est impossible).

3.7 Bilan

Récapitulons ce que nous savons maintenant :

Type	Description
Décidable	Il existe une machine de Turing qui s'arrête toujours et donne la bonne réponse.
Semi-décidable	Il existe une machine qui accepte les mots du langage, mais peut boucler sur les autres. (Elle dit "oui" quand c'est oui, mais ne dit rien quand c'est non.)
Indécidable	Aucune machine de Turing ne peut le décider. C'est une limite *fondamentale* du calcul.

Quiz rapide Bilan du chapitre

1. Le problème de l'arrêt est :
   a) Décidable
   b) Indécidable
   c) Semi-décidable mais pas décidable
2. Si on pouvait décider le problème de l'arrêt, on pourrait aussi décider :

a) Le problème de l'acceptation

b) Tous les problèmes

c) Aucun autre problème

3. La technique de réduction consiste à :

a) Rendre un problème plus petit

b) Montrer qu'un problème est au moins aussi difficile qu'un autre problème connu

c) Transformer une machine non déterministe en déterministe

Réponse

1. **c)** Le problème de l'arrêt est semi-décidable (on peut simuler la machine et dire "oui" si elle s'arrête) mais pas décidable (on ne peut pas dire "non" de manière fiable quand elle ne s'arrête pas).
2. **a)** On a montré par réduction que le problème de l'acceptation se réduit au problème de l'arrêt. Mais pas *tous* les problèmes !
3. **b)** On montre que si le nouveau problème était décidable, alors un problème connu comme indécidable le serait aussi.

Question de réflexion

Le problème de l'arrêt est indécidable dans le cas *général* (pour n'importe quel programme et n'importe quelle entrée). Mais en pratique, pour des programmes spécifiques et bien structurés, on peut souvent prouver qu'ils s'arrêtent. Les outils de vérification formelle font exactement cela. Est-ce une contradiction avec le théorème ? Pourquoi ?

3.8 Exercices

Exercice 1 Décidabilité et labyrinthes.

On considère un labyrinthe fini représenté par un graphe $G = (V, E)$ où V est l'ensemble des cases libres et E les arêtes entre cases adjacentes. Soit S la case de départ et F la case d'arrivée.

a) Le problème "existe-t-il un chemin de S à F dans G ?" est-il décidable ? Justifiez.

b) Proposez un algorithme qui résout ce problème et analysez sa complexité.

c) Et si le labyrinthe était **infini** (un graphe avec un nombre infini de sommets) ? Le problème serait-il toujours décidable ?

Exercice 2 Théorème de Rice en pratique.

Pour chacune des propriétés suivantes, dites si elle est décidable ou indécidable, et justifiez (en utilisant le théorème de Rice si applicable) :

a) "Le programme P contient exactement 3 boucles `for`."

b) "Le programme P accepte le mot vide ε."

c) "Le programme P s'arrête sur toutes les entrées."

d) "Le programme P a un code source de moins de 1000 caractères."

Exercice 3 Réduction.

Montrez que le problème suivant est indécidable : "Étant donné une machine de Turing M, est-ce que $L(M) = \emptyset$ (la machine n'accepte aucun mot) ?"

Indication : réduisez le problème de l'acceptation à ce problème. Construisez, à partir de (M, ω), une machine M' telle que $L(M') = \emptyset$ si et seulement si M n'accepte pas ω.

Exercice 4 Cassez le théorème de Rice ! ★

Pour chacune des propriétés suivantes d'un programme P, dites si le théorème de Rice s'applique (et donc si la propriété est indécidable) ou non. Justifiez.

a) "$L(P)$ contient au moins un mot de longueur paire."

b) "Le code source de P contient la chaîne `"hello"`."

c) "P s'exécute en temps polynomial sur toutes les entrées."

d) "Le code source de P est un palindrome."

e) "$L(P) = \{0^n 1^n \mid n \geq 0\}$."

Indice : les propriétés *sémantiques* (portant sur le langage reconnu, le comportement) sont indécidables par Rice. Les propriétés *syntaxiques* (portant sur le code source) sont décidables.

La suite... On sait maintenant que certains problèmes sont indécidables. Mais parmi les problèmes décidables, sont-ils tous **aussi faciles** les uns que les autres ? Spoiler : absolument pas. Au chapitre suivant, nous allons mesurer la **difficulté** des problèmes décidables et découvrir les classes de complexité.

4 Classes de complexité (Tous les algorithmes ne se valent pas)

Un peu d'histoire

Bon, résumons ce qu'on sait. Il existe des problèmes indécidables : fin de la discussion, aucun algorithme ne peut les résoudre. Mais parmi les problèmes décidables ; ceux qu'on *peut* résoudre ; il y a une question : **à quelle vitesse peut-on les résoudre ?**
Cette question a été formalisée pour la première fois par Juris **Hartmanis** et Richard **Stearns** en **1965**, dans un article fondateur intitulé "On the Computational Complexity of Algorithms". C'est cet article qui a donné son nom au domaine : la **complexité computationnelle**. La même année, Jack **Edmonds** introduit la notion de "bon algorithme" (temps polynomial) et Alan **Cobham** définit la classe $\mathcal{P}$. Ces travaux parallèles établissent le consensus que "polynomial = raisonnable".

À retenir

Objectifs du chapitre :

- Comprendre les notions de **complexité en temps** et **en espace**.
- Maîtriser la notation $O(\cdot)$ et l'analyse asymptotique.
- Découvrir les classes $\mathcal{P}$, $\mathcal{NP}$, PSPACE et EXPTIME.
- Comprendre le **théorème de Savitch** et la hiérarchie des classes.

4.1 Pourquoi mesurer la complexité ?

À vous de jouer !

Imaginez deux algorithmes pour trier une liste de n éléments :

- L'algorithme A fait n^2 opérations.
- L'algorithme B fait $n \log n$ opérations.

Pour $n = 10$, la différence est minime (100 vs 33). Mais pour $n = 1\,000\,000$? A fait 10^{12} opérations (un billion !), B en fait environ 20×10^6 (vingt millions). B est **50 000 fois plus rapide** que A.

La **complexité** d'un algorithme mesure les ressources qu'il consomme en fonction de la taille de l'entrée. On s'intéresse principalement à deux ressources :

- Le **temps** : combien d'étapes de calcul ?
- L'**espace** : combien de cases mémoire ?

4.2 Calculer la complexité

En pratique, on ne traduit pas chaque algorithme en machine de Turing pour compter ses étapes. On identifie des **opérations fondamentales** et on les compte.

À retenir

Pour un problème donné, une **opération fondamentale** est une opération dont le nombre est **proportionnel** au temps d'exécution total de l'algorithme. Le choix dépend du problème :

- **Recherche** dans un tableau → nombre de **comparaisons**.
- **Tri** d'une liste → nombre de **comparaisons** (et/ou échanges).
- **Multiplication** de matrices → nombre de **multiplications scalaires**.

4.3 La notation grand-O

Les fonctions qui comptent le nombre d'opérations en fonction de la taille de l'entrée peuvent être très compliquées. On ne veut pas connaître le nombre *exact* d'opérations; on veut savoir comment ça *évolue* quand n grandit.

À retenir

f est **dominée asymptotiquement** par g, noté $f = O(g)$, si :

$$\exists c \in \mathbb{R}^{+*},\ \exists n_0 \in \mathbb{N},\ \forall n > n_0,\ f(n) \leq c \cdot g(n)$$

En clair : à partir d'un certain rang, $f(n)$ ne dépasse jamais $g(n)$ fois une constante.

Exemple 4.1

- $3n^2+5n+7 = O(n^2)$ *(les termes de degré inférieur et les constantes "disparaissent")*
- $4\log(n) = O(n)$ *(le log est dominé par n'importe quel polynôme)*
- $2^n = O(3^n)$ *(mais $2^n \neq O(n^{100})$!)*

Pause café

Pensez à la notation O comme à "l'ordre de grandeur". Quand vous dites "le restaurant est à 5 minutes", vous ne voulez pas dire exactement 300 secondes. Vous voulez dire que c'est de l'ordre de quelques minutes, pas de quelques heures. La notation O fait la même chose pour les algorithmes.

On utilise aussi les notations Ω et Θ pour des comparaisons plus fines :

À retenir

$f = \Omega(g)$ ("f est **au moins** de l'ordre de g") si :

$$\exists c \in \mathbb{R}^{+*},\ \exists n_0 \in \mathbb{N},\ \forall n > n_0,\ f(n) \geq c \cdot g(n)$$

En clair : à partir d'un certain rang, $f(n)$ ne descend jamais en dessous d'un multiple de $g(n)$.

À vous de jouer !

Pourquoi les constantes n'ont-elles pas d'importance ? Parce qu'un algorithme en $3n^2$ et un autre en $100n^2$ ont le même *comportement asymptotique* : quand n double, le temps est multiplié par 4 dans les deux cas. La constante ne change pas la *nature* de la croissance.
En revanche, n^2 et 2^n ont des comportements fondamentalement différents : quand n double, n^2 est multiplié par 4, mais 2^n est élevé *au carré*. C'est une différence qualitative, pas quantitative.

4.3.1 Règles de comptage

4.3.1.0.1 Séquence. Les nombres d'opérations s'additionnent.

```
a = 2          // 1 affectation
b = 5 + a      // 1 affectation + 1 addition
c = a + b + 2  // 1 affectation + 2 additions
```

Total : 3 affectations, 3 additions. Indépendant de $n \Rightarrow O(1)$.

4.3.1.0.2 Boucle simple.

```
pour i de 1 à n:
    si tab[i] == x:
        retourner i
```

Dans le pire cas, on fait n comparaisons $\Rightarrow O(n)$.

4.3.1.0.3 Boucles imbriquées.

```
pour i de 1 à n:
    pour j de 1 à n:
        si tab[i] > tab[j]:
            échanger(tab[i], tab[j])
```

On fait $n \times n = n^2$ comparaisons $\Rightarrow O(n^2)$.

4.3.1.0.4 Boucle logarithmique.

```
tant que n > 1:
    n = n / 2
```

On divise n par 2 à chaque itération. On fait $\log_2(n)$ itérations $\Rightarrow O(\log n)$.

Exemple 4.2
Recherche dichotomique *dans un tableau trié de n éléments : à chaque étape, on compare x à l'élément central et on élimine la moitié du tableau. Au pire, $\lceil \log_2(n) \rceil$ comparaisons $\Rightarrow O(\log n)$.*

Quiz rapide

Quelle est la complexité du code suivant ?

```
pour i de 1 à n:
    pour j de 1 à i:
        print(i, j)
```

a) $O(n)$

b) $O(n \log n)$

c) $O(n^2)$

d) $O(n^3)$

c). La boucle interne fait i itérations. Le total est $\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = O(n^2)$.

4.3.2 Comparaison d'algorithmes

La notation O donne une **borne supérieure**. Pour comparer plus finement deux algorithmes, on utilise aussi la notation Θ :

À retenir

$f = \Theta(g)$ ("f est du même ordre de grandeur que g") si :

$$\exists c_1, c_2 \in \mathbb{R}^{+*},\ \exists n_0 \in \mathbb{N},\ \forall n > n_0,\ c_2 \cdot g(n) \leq f(n) \leq c_1 \cdot g(n)$$

Θ est plus précis que O : il **encadre** f (au lieu de juste la majorer). Si $f = \Theta(g)$, on dit que f et g ont le même **comportement asymptotique**.

À vous de jouer !

En résumé : $f = O(g)$ dit "f ne dépasse pas g" (c'est une inégalité). $f = \Theta(g)$ dit "f se comporte comme g" (c'est une égalité d'ordre de grandeur).
Analogie : si $f = O(g)$ équivaut à $a \leq b$, alors $f = \Theta(g)$ équivaut à $a = b$ (à des constantes près).

Exemple 4.3
Deux algorithmes de tri comparés :

Algorithme	***Meilleur cas***	***Pire cas***	***En moyenne***
Tri par insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Tri fusion	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$

Pour $n = 10\,000$ sur un ordinateur à 10^9 opérations/seconde :

- *Tri par insertion (pire cas) :* 10^8 *opérations* $\approx 0,1$ *s*
- *Tri fusion (pire cas) :* $\approx 1,3 \times 10^5$ *opérations* $\approx 0,00013$ *s*

Le tri fusion est environ ***770 fois plus rapide*** *dans le pire cas. Et l'écart grandit avec n.*

4.4 Complexité d'une machine de Turing

4.4.1 Compter les étapes et les cases

Pour une machine de Turing M et un mot d'entrée ω :

- $c_t(\omega)$ = nombre d'**étapes de calcul** pour traiter ω.
- $c_e(\omega)$ = nombre de **cases du ruban** utilisées pour traiter ω.

À vous de jouer !

Mais un même algorithme peut mettre plus ou moins de temps selon l'entrée ! Trier une liste déjà triée, c'est rapide. Trier une liste dans le désordre total, c'est long. Comment résumer ça en un seul chiffre ?

On regroupe les entrées par **taille** $n = |\omega|$ (nombre de symboles du mot) et on prend le **pire cas** :

À retenir

- $C_t(n) = \sup\{c_t(\omega) : |\omega| = n\}$
  (complexité en **temps** dans le pire cas pour des entrées de taille n)
- $C_e(n) = \sup\{c_e(\omega) : |\omega| = n\}$
  (complexité en **espace** dans le pire cas pour des entrées de taille n)

À retenir

Relation fondamentale entre temps et espace : pour toute machine de Turing et toute entrée de taille n :

$$C_e(n) \leq C_t(n) + n$$

L'espace utilisé ne peut pas dépasser le nombre d'étapes de calcul plus la taille de l'entrée. La raison est simple : à chaque étape, la tête peut visiter au plus une nouvelle case du ruban.

4.4.2 Et pour les machines non déterministes ?

Pour une machine **non déterministe**, on considère le chemin de calcul **le plus court** (en temps) ou **le moins gourmand** (en espace) qui mène à l'acceptation. C'est le point de vue *optimiste* : on regarde le meilleur choix possible parmi tous les chemins.

4.5 Les grandes classes de complexité

On peut maintenant classer les problèmes selon les ressources nécessaires pour les résoudre.

4.5.1 Les quatre familles

À retenir

Soit f une fonction "raisonnable" :

- DTime(f) : problèmes résolubles en temps $O(f(n))$ par une machine **déterministe**.
- DSpace(f) : problèmes résolubles en espace $O(f(n))$ par une machine **déterministe**.
- NTime(f) : problèmes résolubles en temps $O(f(n))$ par une machine **non déterministe**.
- NSpace(f) : problèmes résolubles en espace $O(f(n))$ par une machine **non déterministe**.

4.5.2 La relation entre déterministe et non déterministe

Un résultat fondamental :

$$\mathrm{DTime}(f(n)) \subseteq \mathrm{NTime}(f(n)) \subseteq \bigcup_{c \geq 1} \mathrm{DTime}(c^{f(n)})$$

En français :

1. Tout ce qu'une machine déterministe fait en temps $f(n)$, une machine non déterministe peut le faire aussi (trivial : une machine déterministe *est* une machine non déterministe qui n'utilise pas le non-déterminisme).

2. Tout ce qu'une machine non déterministe fait en temps $f(n)$, une machine déterministe peut le simuler, mais potentiellement en temps **exponentiel**.

À vous de jouer !

Mais la question brûlante est : ce ralentissement exponentiel est-il **inévitable** ? Ou peut-on faire mieux ? Spoiler : c'est exactement la question P vs NP.

4.5.3 $\mathcal{P}$ et $\mathcal{NP}$

À retenir

- $\mathcal{P} = \bigcup_{c \in \mathbb{R}^+} \mathrm{DTime}(n^c)$

  Les problèmes de classe $\mathcal{P}$ sont ceux résolubles en temps **polynomial** par une machine **déterministe**. Ce sont les problèmes "**traitables**"; ceux pour lesquels on a un algorithme efficace.

- $\mathcal{NP} = \bigcup_{c \in \mathbb{R}^+} \mathrm{NTime}(n^c)$

  Les problèmes de classe $\mathcal{NP}$ sont ceux résolubles en temps **polynomial** par une machine **non déterministe**.

Pause café

Voici une façon plus intuitive de comprendre $\mathcal{NP}$:
Un problème est dans $\mathcal{NP}$ si, quand quelqu'un vous donne une "solution candidate", vous pouvez **vérifier** rapidement (en temps polynomial) si c'est vraiment une solution. Résoudre un Sudoku est difficile (vous devez chercher). Mais **vérifier** qu'une grille remplie est correcte, c'est facile : il suffit de vérifier chaque ligne, colonne et carré. Pas besoin de génie, juste de méthode.
$\mathcal{P}$ = facile à **trouver** la solution.
$\mathcal{NP}$ = facile à **vérifier** une solution.

À retenir

Définition formelle de $\mathcal{NP}$ par vérificateur :
Un langage L est dans $\mathcal{NP}$ s'il existe un **vérificateur** V à temps polynomial et une constante k tels que, pour tout mot x :

$$x \in L \Leftrightarrow \exists c \in \Sigma^*,\ |c| \leq |x|^k,\ V(x,c) \text{ accepte}$$

Le mot c est appelé un **certificat** (ou **témoin**). C'est une "preuve" que $x \in L$, vérifiable en temps polynomial.

À vous de jouer !

Cette définition est équivalente à la définition par machine non déterministe, mais elle est souvent plus pratique. Pour montrer qu'un problème est dans $\mathcal{NP}$, il suffit de :

1. Dire ce qu'est le certificat (la "solution candidate").
2. Montrer qu'il est de taille polynomiale.
3. Montrer qu'on peut le vérifier en temps polynomial.

Est-ce que tout ce qui est facile à vérifier est aussi facile à trouver ? C'est la question à un million de dollars. Littéralement.

4.5.4 co-$\mathcal{NP}$

À retenir

Un langage L est dans **co-$\mathcal{NP}$** si son complémentaire $\overline{L}$ est dans $\mathcal{NP}$.
Autrement dit : $L \in$ co-$\mathcal{NP}$ si on peut **réfuter** efficacement , s'il existe un certificat polynomial prouvant que $x \notin L$.

Exemple 4.4

- ***SAT*** *est dans $\mathcal{NP}$: le certificat est une affectation satisfaisante.*
- ***UNSAT*** *("la formule n'est satisfaisable par* aucune *affectation") est dans co-$\mathcal{NP}$: pour réfuter, il suffit de donner une affectation satisfaisante (certificat pour SAT).*
- ***TAUTOLOGIE*** *("la formule est vraie pour* toute *affectation") est dans co-$\mathcal{NP}$.*

À vous de jouer !

On sait que $\mathcal{P} \subseteq \mathcal{NP} \cap$ co-$\mathcal{NP}$ (tout langage décidable en temps polynomial est à la fois "vérifiable" et "réfutable"). Mais on ne sait pas si $\mathcal{NP} =$ co-$\mathcal{NP}$. Si $\mathcal{P} = \mathcal{NP}$, alors $\mathcal{NP} =$ co-$\mathcal{NP}$. Mais la réciproque n'est pas connue !

4.5.5 Pour bien comprendre : un exemple avec le Sudoku

Le **Sudoku** est un problème de la classe $\mathcal{NP}$. Pourquoi ?

4.5.5.0.1 Algorithme non déterministe (version $\mathcal{NP}$) :

1. "Deviner" (non déterminisme) une valeur pour chaque case vide.
2. Vérifier que la grille obtenue respecte les règles.

La vérification se fait en $O(n^3)$ comparaisons (vérifier chaque ligne, colonne et carré). C'est polynomial. Donc le Sudoku $\in \mathcal{NP}$.

4.5.5.0.2 Algorithme déterministe (version brute force) : Explorer **toutes** les façons de remplir la grille. Il y a au plus n^{n^2} grilles possibles. Pour chacune, la vérification prend $O(n^3)$. Total : $O(n^{n^2+3})$. C'est **exponentiel** et inutilisable en pratique.

À vous de jouer !

Cela illustre parfaitement l'écart entre $\mathcal{P}$ et $\mathcal{NP}$: vérifier une solution de Sudoku est facile (polynomial), mais **trouver** une solution semble beaucoup plus difficile.

4.5.6 La thèse de l'invariance

À retenir

La traduction d'un programme réel en une machine de Turing introduit au plus un **ralentissement polynomial**. La classe $\mathcal{P}$ est donc **stable** quel que soit le modèle de calcul choisi (machine de Turing, programme C, programme Python...).

Cela signifie que quand on dit "un problème est dans $\mathcal{P}$", cette affirmation ne dépend *pas* du langage de programmation ou du modèle d'ordinateur. C'est une propriété *intrinsèque* du problème.

4.6 La hiérarchie des vitesses

Pour mettre les choses en perspective, voici un tableau qui montre le temps de calcul pour différentes complexités, avec un ordinateur effectuant 10^9 opérations par seconde :

Complexité	$n = 10$	$n = 50$	$n = 100$	$n = 1000$
n	10^{-8} s	5×10^{-8} s	10^{-7} s	10^{-6} s
n^2	10^{-7} s	$2,5 \times 10^{-6}$ s	10^{-5} s	10^{-3} s
n^3	10^{-6} s	$1,25 \times 10^{-4}$ s	10^{-3} s	1 s
2^n	10^{-6} s	13 jours	4×10^{13} ans	☠
$n!$	3,6 ms	10^{48} ans	☠	☠

Pause café

Vous voyez le problème ? Pour les complexités polynomiales (n, n^2, n^3), tout va bien. Les temps de calcul restent raisonnables même pour de grandes entrées.
Mais dès qu'on passe en **exponentiel** (2^n) ou en **factoriel** ($n!$), c'est la catastrophe. Pour $n = 100$, 2^n opérations prendraient plus de temps que l'âge de l'Univers. Et $n!$ pour $n = 50$? On parle de 10^{48} ans. L'Univers n'aura probablement même plus d'étoiles à ce moment-là.
C'est pourquoi la frontière polynomial/exponentiel est si importante.

4.6.1 Le théorème de hiérarchie temporelle

Ce tableau ne fait qu'illustrer une vérité profonde, prouvée rigoureusement par Hartmanis et Stearns en 1965 :

À retenir

Théorème de la hiérarchie temporelle : Si $f(n) \cdot \log^2(f(n)) = o(g(n))$, alors

$$\mathrm{DTime}(f(n)) \subsetneq \mathrm{DTime}(g(n))$$

En clair : **avec plus de temps, on peut résoudre strictement plus de problèmes.** Il existe des problèmes résolubles en temps n^3 qui ne sont *pas* résolubles en temps n^2.

À vous de jouer !

Ce théorème a une conséquence spectaculaire : il prouve que $\mathcal{P} \neq \mathrm{EXPTIME}$ (la classe des problèmes résolubles en temps exponentiel). On sait **avec certitude** qu'il existe des problèmes résolubles en temps exponentiel mais *pas* en temps polynomial. Ce qu'on ne sait pas, c'est *lesquels* (sont-ce les problèmes $\mathcal{NP}$-complets ?).

4.7 Le théorème de Savitch : espace déterministe vs non déterministe

On a vu que simuler une machine non déterministe par une machine déterministe peut nécessiter un temps **exponentiel**. Qu'en est-il de l'**espace** ? La réponse, due à **Walter Savitch** (1970), est bien plus optimiste : le non-déterminisme ne coûte que le *carré* de l'espace.

À retenir

Théorème de Savitch (1970) : Pour toute fonction $s(n) \geq \log n$:

$$\mathrm{NSpace}(s(n)) \subseteq \mathrm{DSpace}(s(n)^2)$$

En particulier :

$$\text{PSPACE} = \text{NPSPACE}$$

L'espace non déterministe polynomial est égal à l'espace déterministe polynomial. Contrairement au temps, le non-déterminisme n'apporte aucun gain exponentiel en espace.

4.7.1 Intuition : la technique de bissection

Comment simuler de façon déterministe un calcul non déterministe avec si peu d'espace supplémentaire ? L'idée clé : au lieu d'explorer les chemins de calcul un par un (ce qui prendrait de l'espace exponentiel pour tout mémoriser), on **bisecte** le calcul.

On se pose la question : "Peut-on passer de la configuration C_1 à la configuration C_2 en au plus t étapes ?" On note cette propriété $\text{Reach}(C_1, C_2, t)$.

À retenir

Récursion de bissection :

$$\text{Reach}(C_1, C_2, t) \;\Leftrightarrow\; \exists\, C_m :\; \text{Reach}(C_1, C_m, \lceil t/2 \rceil) \wedge \text{Reach}(C_m, C_2, \lfloor t/2 \rfloor)$$

On cherche une configuration intermédiaire C_m au milieu du chemin de calcul. Si elle existe, on résout récursivement les deux moitiés.

Analyse de l'espace :

- La récursion a une profondeur $\log_2(t)$ niveaux.
- À chaque niveau, on mémorise trois configurations (C_1, C_m, C_2), soit $O(s(n))$ espace par niveau.
- Espace total : $O(s(n) \cdot \log t)$.

Le nombre maximal de configurations distinctes d'une machine utilisant $s(n)$ espace est $2^{O(s(n))}$, donc $t \leq 2^{O(s(n))}$ et $\log t = O(s(n))$. L'espace total est $O(s(n)^2)$. C'est bien la borne du théorème.

Pause café

Imaginez un GPS non déterministe qui "essaie" toutes les routes simultanément. Une approche naïve mémoriserait *tous* les chemins possibles ; un espace exponentiel. Savitch dit : mémorisez seulement le point de départ, la destination, et la longueur maximale du trajet. Puis cherchez un point intermédiaire, récursivement. Chaque sous-problème est de moitié plus petit. L'espace utilisé est **quadratique** en la taille du problème, jamais exponentiel.
L'astuce est qu'on peut *réutiliser l'espace* d'un appel récursif à l'autre (effacer et réécrire), alors que le temps, une fois consommé, est perdu.

4.7.2 Conséquences et hiérarchie complète

À retenir

Hiérarchie complète des classes de complexité :

$$\mathcal{P} \subseteq \mathcal{NP} \subseteq \text{PSPACE} = \text{NPSPACE} \subseteq \text{EXPTIME}$$

- $\mathcal{NP} \subseteq$ PSPACE : l'exploration de l'arbre de calcul non déterministe peut se faire en DFS avec une pile de profondeur $p(n)$, espace polynomial.
- PSPACE = NPSPACE : **théorème de Savitch**. Le non-déterminisme ne coûte qu'un facteur quadratique.
- PSPACE $\subseteq$ EXPTIME : en espace polynomial n^k, il y a au plus $2^{O(n^k)}$ configurations distinctes, donc le calcul s'arrête en temps exponentiel.

On sait que $\mathcal{P} \neq$ EXPTIME (théorème de hiérarchie temporelle). Mais on ne sait pas si les inclusions intermédiaires sont strictes. Est-ce que $\mathcal{NP} \subsetneq$ PSPACE ? Mystère.

À vous de jouer !

Pourquoi le théorème de Savitch ne s'applique-t-il **pas** au *temps* ? La technique de bissection mémorise les configurations intermédiaires, ce qui réutilise l'*espace*. Mais pour le temps, chaque appel récursif doit *recalculer* les sous-problèmes depuis zéro (on ne peut pas "revenir en arrière dans le temps"). C'est une raison profonde pour laquelle l'espace et le temps se comportent différemment vis-à-vis du non-déterminisme.

Exemple 4.5
PSPACE-complet : le problème QBF.

*La **formule booléenne quantifiée** (Quantified Boolean Formula) est une formule de la forme :*

$$\forall x_1 \, \exists x_2 \, \forall x_3 \, \cdots \, \varphi(x_1, \ldots, x_n)$$

*On demande si elle est vraie. Ce problème est **PSPACE-complet** : il est le "SAT de PSPACE". Si SAT est la question canonique de $\mathcal{NP}$, QBF est la question canonique de* PSPACE. *En pratique, QBF est au cœur de la **vérification formelle** (model checking), de la planification en IA, et de l'analyse de protocoles cryptographiques.*

4.8 Exercices

Exercice 1 Comptage d'opérations.

Pour chacun des algorithmes suivants, déterminez la complexité en fonction de n (nombre d'éléments) pour l'opération fondamentale indiquée.

a) Recherche séquentielle d'un élément dans un tableau non trié de n éléments. Opération : comparaison.

b) Calcul de $n!$ par multiplications successives. Opération : multiplication.

c) Vérification qu'un tableau de n éléments ne contient pas de doublons (par double boucle). Opération : comparaison.

Exercice 2 BFS dans un labyrinthe.

On considère le BFS (parcours en largeur) dans un labyrinthe $n \times n$:

```
BFS(G, S, F):
    file = [S]
    marque[S] = vrai
    tant que file non vide:
        u = retirer(file)
        si u == F: retourner "trouvé"
        pour v adjacent à u:
            si non marque[v]:
                marque[v] = vrai
                ajouter(file, v)
    retourner "pas de chemin"
```

a) Identifiez les opérations fondamentales.

b) Montrez que la complexité est $O(|V| + |E|)$ où $|V|$ est le nombre de cases libres et $|E|$ le nombre d'arêtes.

c) Pour un labyrinthe $n \times n$, quel est l'ordre de grandeur de $|V|$ et $|E|$ dans le pire cas ?

d) Comparez avec un algorithme qui testerait tous les chemins possibles (exploration exhaustive).

Exercice 3 Comparaison.

On dispose de deux algorithmes pour résoudre le même problème :

- Algorithme A : $f(n) = 100n^2$ opérations.
- Algorithme B : $g(n) = 2n^3$ opérations.

a) Pour quelles valeurs de n l'algorithme A est-il plus rapide que B ?

b) Donnez les complexités asymptotiques de A et B.

c) Un troisième algorithme C fait 2^n opérations. À partir de quelle valeur de n devient-il plus lent que A ?

4.9 Résumé : la grande image

Tous les problèmes
⊃ **Problèmes décidables** (une machine de Turing peut les résoudre)
⊃ $\mathcal{NP}$ (vérifiables en temps polynomial)
⊃ $\mathcal{P}$ (résolubles en temps polynomial)
⊃ ??? (On ne sait pas si $\mathcal{P} = \mathcal{NP}$ ou $\mathcal{P} \subsetneq \mathcal{NP}$)

Quiz rapide Bilan du chapitre

1. $3n^3 + 100n^2 + 5 = ?$
   a) $O(n^2)$
   b) $O(n^3)$
   c) $O(n^4)$
   d) b) et c) sont tous les deux corrects
2. Un problème dans $\mathcal{P}$ est aussi dans $\mathcal{NP}$?
   a) Toujours
   b) Jamais
   c) Ça dépend
3. Un problème dans $\mathcal{NP}$ peut être résolu en temps polynomial par une machine déterministe ?
   a) Toujours
   b) On ne sait pas (c'est la question P vs NP)
   c) Jamais
4. Un algorithme en $O(2^n)$ est :
   a) Polynomial
   b) Exponentiel
   c) Linéaire

Réponse

1. **d)** $O(n^3)$ est la borne la plus précise, mais $O(n^4)$ est aussi techniquement correct (si $f = O(n^3)$, alors $f = O(n^4)$ aussi). En pratique, on cherche la borne la plus serrée, donc $O(n^3)$.
2. **a)** Oui, toujours ! $\mathcal{P} \subseteq \mathcal{NP}$. Une machine déterministe est un cas particulier de machine non déterministe.
3. **b)** C'est exactement la question P vs NP ! On ne sait pas si tout problème $\mathcal{NP}$ peut être résolu efficacement de manière déterministe.
4. **b)** Exponentiel.

Question de réflexion

Existe-t-il des problèmes qui sont dans $\mathcal{NP}$ mais dont on a prouvé qu'ils ne sont *pas* dans $\mathcal{P}$? (Indice : la réponse est liée à la question P vs NP.) Et si on n'arrive pas à prouver que $\mathcal{P} \neq \mathcal{NP}$, qu'est-ce que cela signifierait ?

La suite... On a posé les classes de complexité et leur hiérarchie. Mais laquelle de ces inclusions est stricte ? La question $\mathcal{P} \stackrel{?}{=} \mathcal{NP}$ est la plus célèbre et elle vaut un million de dollars. C'est le sujet du chapitre final.

5 $\mathcal{P}$ vs $\mathcal{NP}$

Un peu d'histoire

L'an 2000. Le millénaire change. L'Institut Clay de mathématiques publie une liste de sept problèmes mathématiques, les "Problèmes du Millénaire", et offre **un million de dollars** pour la résolution de chacun.
Depuis, un seul a été résolu : la conjecture de Poincaré, par le mathématicien russe Grigori Perelman en 2003 (qui a *refusé* le prix, parce que les mathématiciens sont des gens spéciaux :-)).
Parmi les six restants, l'un est au cœur de l'informatique : **P vs NP**. Et c'est le sujet de ce chapitre.
La question peut se formuler simplement :

Si on peut vérifier rapidement qu'une solution est correcte,
peut-on aussi la ***trouver*** *rapidement ?*

Simple à énoncer. Impossible (pour l'instant) à résoudre.

À retenir

Objectifs du chapitre :

- Comprendre la notion de $\mathcal{NP}$**-complétude** et sa signification.
- Connaître le **théorème de Cook-Levin** et les réductions polynomiales.
- Découvrir les principaux problèmes $\mathcal{NP}$-complets.
- Savoir quelles **stratégies pratiques** adopter face à un problème $\mathcal{NP}$-complet.

5.1 Retour sur $\mathcal{P}$ et $\mathcal{NP}$

Rappelons les définitions.

À retenir

- $\mathcal{P}$ = l'ensemble des problèmes qu'on peut **résoudre** en temps polynomial (par une machine déterministe).
- $\mathcal{NP}$ = l'ensemble des problèmes qu'on peut **vérifier** en temps polynomial. Plus formellement : les problèmes résolubles en temps polynomial par une machine non déterministe.

On sait que $\mathcal{P} \subseteq \mathcal{NP}$ (tout ce qu'on peut résoudre vite, on peut aussi le vérifier vite). La grande question :

$$\boxed{\mathcal{P} \stackrel{?}{=} \mathcal{NP}}$$

À vous de jouer !

Traduction concrète : existe-t-il des problèmes dont la solution est facile à **vérifier** mais fondamentalement difficile à **trouver** ?
Presque tout le monde pense que oui ($\mathcal{P} \neq \mathcal{NP}$). Mais personne n'a réussi à le prouver.

5.1.1 Pourquoi c'est important

La résolution de P vs NP aurait des conséquences pratiques **énormes** :

- **Cryptographie.** La sécurité de vos mots de passe, de vos transactions bancaires, de vos messages chiffrés repose sur l'hypothèse que certains problèmes sont *difficiles* à résoudre (mais faciles à vérifier). Si $\mathcal{P} = \mathcal{NP}$, toute la cryptographie moderne s'effondrerait.
- **Optimisation.** Des milliers de problèmes industriels (logistique, emploi du temps, planification) sont dans $\mathcal{NP}$. Si $\mathcal{P} = \mathcal{NP}$, on pourrait tous les résoudre efficacement.
- **Intelligence artificielle.** De nombreux problèmes d'IA sont dans $\mathcal{NP}$. Leur résolution efficace révolutionnerait le domaine.
- **Mathématiques elles-mêmes.** Si $\mathcal{P} = \mathcal{NP}$, on pourrait *trouver* des preuves mathématiques aussi facilement qu'on peut les *vérifier*. Les mathématiciens seraient (partiellement) remplacés par des ordinateurs.

Pause café

Scott Aaronson, un célèbre informaticien théoricien, a dit un jour : "Si $\mathcal{P} = \mathcal{NP}$, alors le monde serait un endroit profondément différent de ce que nous pensons habituellement. Il n'y aurait aucune valeur spéciale dans la 'perspicacité créative' ; tout ce qui pourrait être reconnu efficacement pourrait aussi être produit efficacement." En d'autres termes : si trouver était aussi facile que vérifier, alors *créer* serait aussi facile que *critiquer*. Tout critique d'art serait automatiquement capable de peindre

la Joconde. Tout lecteur de roman serait capable d'écrire le prochain prix Goncourt. Ça semble... improbable, non ?

5.2 Les problèmes $\mathcal{NP}$-complets

Parmi tous les problèmes $\mathcal{NP}$, certains sont plus intéressants que d'autres : les problèmes **$\mathcal{NP}$-complets**. Ce sont les problèmes les **plus difficiles** de $\mathcal{NP}$.

5.2.1 Transformations polynomiales (réductions)

À retenir

Une **transformation polynomiale** (ou réduction polynomiale) de L_1 vers L_2 est une fonction f qui :

- est calculable en temps polynomial,
- vérifie : $x \in L_1 \Leftrightarrow f(x) \in L_2$.

On note $L_1 \leq_p L_2$ ("L_1 se réduit polynomialement à L_2").

Pause café

Une réduction polynomiale, c'est comme un traducteur. Vous avez un problème en français (L_1) et un solveur qui ne parle que l'anglais (L_2). Si vous pouvez **traduire rapidement** votre problème en anglais (en temps polynomial), et si le solveur anglais donne la bonne réponse, alors vous pouvez résoudre votre problème français.
La conséquence : si le problème français est difficile, le problème anglais l'est **au moins autant** (sinon la traduction + résolution serait une méthode facile pour résoudre le problème français).

Propriétés :

- Si $L_1 \leq_p L_2$ et $L_2 \in \mathcal{P}$, alors $L_1 \in \mathcal{P}$.
- Si $L_1 \leq_p L_2$ et $L_1 \notin \mathcal{P}$, alors $L_2 \notin \mathcal{P}$.

La seconde propriété est la contraposée de la première. C'est la puissance de la réduction : elle permet de "transférer" la difficulté d'un problème à un autre.

5.2.2 Définition formelle de $\mathcal{NP}$-complet

À retenir

Un langage L est **$\mathcal{NP}$-complet** si :

1. $L \in \mathcal{NP}$ (le problème est bien dans $\mathcal{NP}$),
2. **Tout** langage $L' \in \mathcal{NP}$ se réduit polynomialement à L ($L' \leq_p L$).

À vous de jouer !

La condition 2 est importante : un problème $\mathcal{NP}$-complet est un problème auquel **tout** problème de $\mathcal{NP}$ peut se ramener ! C'est le "boss de fin de niveau" de $\mathcal{NP}$: si vous arrivez à le battre (le résoudre en temps polynomial), alors vous pouvez battre **tous** les problèmes de $\mathcal{NP}$.

À retenir

Conséquence cruciale : S'il existe un **seul** problème $\mathcal{NP}$-complet qui est dans $\mathcal{P}$, alors $\mathcal{P} = \mathcal{NP}$.
Réciproquement, si $\mathcal{P} \neq \mathcal{NP}$, alors **aucun** problème $\mathcal{NP}$-complet n'est dans $\mathcal{P}$.

5.2.3 $\mathcal{NP}$-difficile vs $\mathcal{NP}$-complet

À retenir

Un problème est **$\mathcal{NP}$-difficile** (ou **$\mathcal{NP}$-dur**, *NP-hard*) si tout problème de $\mathcal{NP}$ se réduit polynomialement à lui mais il n'est **pas forcément** dans $\mathcal{NP}$ lui-même.
Un problème est $\mathcal{NP}$-complet s'il est $\mathcal{NP}$-difficile **et** dans $\mathcal{NP}$.

$$\text{NP-complet} = \text{NP-difficile} \cap \mathcal{NP}$$

À vous de jouer !

La distinction est subtile mais importante :

- Le **problème de l'arrêt** est $\mathcal{NP}$-difficile (tout problème de $\mathcal{NP}$ s'y réduit), mais il n'est **pas** dans $\mathcal{NP}$ (il n'est même pas décidable !). Donc il n'est pas $\mathcal{NP}$-complet.
- Le **voyageur de commerce** sous sa forme d'optimisation ("trouver le *plus court* tour") est $\mathcal{NP}$-difficile, mais ce n'est pas un problème de décision (oui/non), donc on ne peut pas dire qu'il est dans $\mathcal{NP}$ au sens strict. Sa version décisionnelle ("existe-t-il un tour de longueur $\leq B$?") est, elle, $\mathcal{NP}$-complète.

5.3 SAT, Le premier problème $\mathcal{NP}$-complet

5.3.1 Rappels de logique

Le calcul propositionnel manipule des **variables booléennes** (vrai ou faux) avec les opérations :

- **Négation** : $\neg p$ (“pas p”)
- **Conjonction** : $p \wedge q$ (“p et q”)
- **Disjonction** : $p \vee q$ (“p ou q”)

Tables de vérité :

p	$\neg p$
vrai	faux
faux	vrai

p	q	$p \wedge q$
V	V	V
V	F	F
F	V	F
F	F	F

p	q	$p \vee q$
V	V	V
V	F	V
F	V	V
F	F	F

5.3.2 Forme normale conjonctive

Une expression est en **forme normale conjonctive (FNC)** si elle est de la forme :

$$C_1 \wedge C_2 \wedge \ldots \wedge C_m$$

où chaque C_i est une **clause** (disjonction de littéraux) :

$$C_i = \ell_1 \vee \ell_2 \vee \ldots \vee \ell_k$$

et chaque ℓ_j est un **littéral** (p ou $\neg p$).

Exemple 5.1
$(p \vee q) \wedge (\neg r) \wedge (\neg p \vee r)$ *est en FNC.*

- *Clause 1 :* $p \vee q$
- *Clause 2 :* $\neg r$
- *Clause 3 :* $\neg p \vee r$

Cette formule est-elle satisfaisable ? Essayons $p =$ *vrai,* $q =$ *vrai,* $r =$ *faux :*

- $p \vee q = V \vee V = V$ ✓
- $\neg r = \neg F = V$ ✓
- $\neg p \vee r = F \vee F = F$ ✗

Raté ! La clause 3 n'est pas satisfaite.
Essayons $p =$ *faux,* $q =$ *vrai,* $r =$ *faux :*

- $p \vee q = F \vee V = V$ ✓
- $\neg r = V$ ✓
- $\neg p \vee r = V \vee F = V$ ✓

Toutes les clauses sont satisfaites ! La formule est ***satisfaisable****.*

5.3.3 Le problème SAT

Problème SAT

Données : Une expression du calcul propositionnel en forme normale conjonctive.

Question : Cette expression est-elle satisfaisable ?

À vous de jouer !

En clair : étant donné un ensemble de contraintes logiques, existe-t-il un moyen de rendre toutes les contraintes vraies en même temps ?

5.3.4 Le théorème de Cook-Levin (1971)

À retenir

Théorème de Cook-Levin : Le problème SAT est $\mathcal{NP}$-complet.

C'est un résultat historique. Il a été prouvé indépendamment par Stephen Cook (1971) et Leonid Levin (1973). Il montre que SAT est "le plus difficile" des problèmes de $\mathcal{NP}$.

Que dit la preuve ? Pour **tout** problème $L \in \mathcal{NP}$ et **tout** mot ω, on peut construire (en temps polynomial) une formule en FNC qui est satisfaisable si et seulement si $\omega \in L$.

À retenir

Esquisse de la preuve de Cook-Levin :
Soit $L \in \mathcal{NP}$. Par définition, il existe une machine de Turing non déterministe M qui décide L en temps $p(n)$ (polynomial). On construit une formule booléenne φ qui *encode le calcul* de M sur ω :

1. **Variables.** On introduit des variables booléennes pour encoder, à chaque instant $t \in \{0, \ldots, p(n)\}$:
   - l'état de la machine : $\text{etat}_{t,q}$ ("la machine est dans l'état q au temps t"),
   - la position de la tête : $\text{tete}_{t,i}$ ("la tête est sur la case i au temps t"),
   - le contenu du ruban : $\text{ruban}_{t,i,a}$ ("la case i contient le symbole a au temps t").
2. **Clauses.** On ajoute des clauses qui garantissent :
   - la configuration initiale est correcte (ω sur le ruban, état q_0, tête en position 0),
   - chaque transition respecte la fonction δ,
   - la machine est dans exactement un état, la tête est en exactement une position, chaque case contient exactement un symbole,

- au temps final, la machine est dans un état acceptant.

3. **Résultat.** φ est satisfaisable $\Leftrightarrow$ il existe un calcul acceptant de M sur $\omega \Leftrightarrow$ $\omega \in L$.

Le nombre de variables et de clauses est **polynomial** en n (car $p(n)$ est polynomial et le nombre d'états et de symboles est constant). La construction est donc polynomiale.

Cela signifie que SAT est un "problème universel" pour $\mathcal{NP}$: tout problème de $\mathcal{NP}$ peut être "traduit" en SAT.

Un peu d'histoire

Stephen Cook a prouvé ce théorème en 1971. Au début, la communauté n'a pas immédiatement saisi la portée du résultat. C'est Richard Karp qui, un an plus tard, a montré que 21 problèmes classiques étaient $\mathcal{NP}$-complets en les réduisant les uns aux autres (et ultimement à SAT). C'est à ce moment-là que le monde de l'informatique théorique a compris l'importance de la NP-complétude.

5.3.5 3-SAT : la variante canonique

En pratique, pour les réductions, on utilise souvent une variante restreinte de SAT :

À retenir

3-SAT est la restriction de SAT aux formules en FNC dont chaque clause contient **exactement 3 littéraux**.
Exemple : $(p \vee \neg q \vee r) \wedge (\neg p \vee q \vee \neg r) \wedge (p \vee q \vee r)$ est une instance de 3-SAT.

À retenir

Théorème : 3-SAT est $\mathcal{NP}$-complet.
Idée de la preuve : on montre que SAT $\leq_p$ 3-SAT. Toute clause de plus de 3 littéraux peut être "cassée" en plusieurs clauses de 3 littéraux en introduisant des variables auxiliaires, de manière à préserver la satisfaisabilité. La construction est polynomiale.

À vous de jouer !

Pourquoi 3-SAT est-il si important ? Parce que ses clauses ont une taille fixe (3 littéraux), ce qui rend les constructions de réductions beaucoup plus maniables. C'est le point de départ de la plupart des preuves de $\mathcal{NP}$-complétude : on réduit 3-SAT (et non SAT directement) au problème cible. C'est par exemple le cas pour les réductions vers Clique, 3-Coloration, ou Ensemble Indépendant mentionnées dans les exercices de ce chapitre.

5.3.6 Pourquoi SAT est dans $\mathcal{NP}$

- **Certificat** : une affectation des variables ($p = V$, $q = F$, etc.).
- **Vérification** : on évalue chaque clause. Si toutes sont vraies, l'affectation satisfait la formule. Temps de vérification : $O(n)$ où n est la taille de la formule.

Facile à vérifier, potentiellement difficile à trouver. C'est le cœur de $\mathcal{NP}$.

Pause café

SAT dans la vraie vie. Les solveurs SAT modernes ne sont pas que des curiosités théoriques. Ils sont utilisés quotidiennement dans l'industrie :

- **Vérification de processeurs** : Intel et AMD utilisent des solveurs SAT pour détecter des bugs matériels *avant* la fabrication des puces. Un bug non détecté peut coûter des milliards (cf. le bug Pentium FDIV en 1994).
- **Planification aérienne** : affecter des équipages, des avions et des créneaux horaires à des milliers de vols sous contraintes se traduit naturellement en un problème SAT.
- **Bioinformatique** : déterminer la structure 3D de protéines sous contraintes physiques peut être encodé comme une instance SAT.

Le fait que SAT soit $\mathcal{NP}$-complet ne signifie pas qu'il est inutilisable : les solveurs actuels résolvent couramment des instances à **des millions de variables**.

5.4 La galaxie des problèmes $\mathcal{NP}$-complets

Depuis le théorème de Cook-Levin, des **milliers** de problèmes ont été identifiés comme $\mathcal{NP}$-complets. En voici quelques-uns parmi les plus célèbres :

5.4.1 Le problème du voyageur de commerce

Problème du voyageur de commerce

Données : n villes et les distances entre elles, un budget B.

Question : Existe-t-il un tour passant par toutes les villes et revenant au point de départ dont la longueur totale est $\leq B$?

Pause café

Imaginez un livreur Amazon qui doit livrer 100 colis dans 100 villes différentes et revenir à l'entrepôt. Trouver le trajet le plus court est un cauchemar algorithmique. Pourtant, si quelqu'un vous *donne* un trajet, vérifier que sa longueur est inférieure à B est trivial : il suffit d'additionner les distances.

5.4.2 Le problème du sac à dos

Problème du sac à dos

Données : n objets avec chacun un poids et une valeur, et une capacité maximale W.

Question : Peut-on choisir des objets dont la valeur totale est $\geq V$ et le poids total $\leq W$?

5.4.3 La coloration de graphe

Problème de k-coloration

Données : Un graphe G et un entier k.

Question : Peut-on colorier les sommets de G avec k couleurs telles que deux sommets adjacents n'aient jamais la même couleur ?

5.4.4 Le problème de la clique

Problème de la clique

Données : Un graphe G et un entier k.

Question : Existe-t-il un sous-ensemble de k sommets tous connectés deux à deux ?

5.4.5 Comment prouver qu'un problème est $\mathcal{NP}$-complet

Pour montrer qu'un problème L est $\mathcal{NP}$-complet :

1. Montrer que $L \in \mathcal{NP}$ (exhiber un certificat vérifiable en temps polynomial).
2. Trouver un problème L' **déjà connu** comme $\mathcal{NP}$-complet et montrer que $L' \leq_p L$ (réduction polynomiale).

À vous de jouer !

La deuxième étape est souvent la plus créative. Il faut trouver le "bon" problème à réduire et la "bonne" transformation. C'est un art autant qu'une science.

5.4.6 Exemple complet de réduction : Clique $\leq_p$ Ensemble Indépendant

Montrons que le problème de l'**ensemble indépendant** est $\mathcal{NP}$-complet en le réduisant depuis le problème de la **clique** (déjà connu comme $\mathcal{NP}$-complet).

Problème de l'ensemble indépendant

Données : Un graphe $G = (V, E)$ et un entier k.

Question : Existe-t-il un sous-ensemble de k sommets **sans aucune arête entre eux** ?

5.4.6.0.1 Étape 1 : Ensemble Indépendant $\in \mathcal{NP}$. Le certificat est un sous-ensemble S de k sommets. On vérifie en $O(k^2) \leq O(n^2)$ qu'aucune paire de sommets de S n'est reliée par une arête. C'est polynomial.

5.4.6.0.2 Étape 2 : Réduction depuis Clique. Soit (G, k) une instance de Clique. On construit le **graphe complémentaire** $\overline{G} = (V, \overline{E})$, où $\overline{E} = \{(u, v) \mid (u, v) \notin E\}$. Autrement dit, on *inverse* toutes les arêtes : là où il y avait une arête, on l'enlève, et là où il n'y en avait pas, on en ajoute une.

Affirmation : G a une clique de taille k si et seulement si $\overline{G}$ a un ensemble indépendant de taille k.

Preuve :

- ($\Rightarrow$) Si S est une clique dans G : tous les sommets de S sont reliés entre eux dans G, donc **aucun** n'est relié dans $\overline{G}$. Donc S est un ensemble indépendant dans $\overline{G}$.
- ($\Leftarrow$) Inversement : si S est indépendant dans $\overline{G}$, aucune paire n'est reliée dans $\overline{G}$, donc toutes sont reliées dans G : c'est une clique.

Complexité de la réduction : construire $\overline{G}$ prend $O(n^2)$, qui est polynomial. □

Pause café

Cette réduction est élégante car elle est presque triviale : il suffit d'inverser les arêtes ! C'est un excellent premier exemple de réduction pour s'entraîner. Les réductions plus complexes (comme SAT vers 3-coloration) nécessitent des constructions beaucoup plus ingénieuses.

Quiz rapide

Parmi ces problèmes, lequel n'est **pas** $\mathcal{NP}$-complet ?

a) SAT

b) Le tri d'une liste

c) Le problème du voyageur de commerce

d) La 3-coloration de graphe

b) Le tri d'une liste est dans $\mathcal{P}$ (on sait le faire en $O(n \log n)$). Il est donc dans $\mathcal{NP}$ mais n'est pas $\mathcal{NP}$-complet (sauf si $\mathcal{P} = \mathcal{NP}$, ce que personne ne croit).

5.5 Les stratégies face à la $\mathcal{NP}$-complétude

À vous de jouer !

Vous êtes développeur. Votre chef vous demande de résoudre un problème $\mathcal{NP}$-complet. Que faites-vous ?

D'abord : vous lui expliquez poliment que ce n'est pas qu'une question de compétence, c'est un problème fondamental de l'informatique que personne au monde n'a réussi à résoudre efficacement.

(Vous pouvez aussi imprimer le théorème de Cook-Levin, l'encadrer, et le poser sur votre bureau. Quand le chef reviendra demander pourquoi c'est si long, pointez le cadre. 😊)

Ensuite : vous proposez des alternatives :

1. **Algorithmes d'approximation.** On ne cherche pas la solution *optimale*, mais une solution "assez bonne". Par exemple, pour le voyageur de commerce, on peut trouver un trajet qui est au pire 1,5 fois plus long que le trajet optimal.

2. **Heuristiques.** Des méthodes qui marchent "souvent" en pratique, sans garantie théorique. Les algorithmes génétiques, le recuit simulé, les réseaux de neurones entrent dans cette catégorie.

3. **Solveurs SAT.** On traduit le problème en une instance de SAT et on utilise un solveur SAT moderne. Ces solveurs sont extraordinairement efficaces en pratique, même si leur pire cas est exponentiel.

4. **Instances restreintes.** Le problème est peut-être $\mathcal{NP}$-complet dans le cas général, mais vos instances concrètes ont peut-être une structure particulière qui les rend traitables.

5.6 Au-delà de $\mathcal{NP}$: un aperçu

$\mathcal{P}$ et $\mathcal{NP}$ ne sont que la partie émergée de l'iceberg. Il existe **plus de 460 classes de complexité** répertoriées ! En voici quelques-unes :

- **co-$\mathcal{NP}$** : les problèmes dont le *complémentaire* est dans $\mathcal{NP}$. Exemple : prouver qu'une formule *n'est pas* satisfaisable.

- **PSPACE** : les problèmes résolubles avec une quantité **polynomiale de mémoire** (mais potentiellement un temps exponentiel). On sait que $\mathcal{NP} \subseteq$ PSPACE.

- **EXPTIME** : les problèmes résolubles en temps exponentiel. On sait que PSPACE $\subseteq$ EXPTIME.
- **BQP** : les problèmes résolubles efficacement par un **ordinateur quantique**. Le rapport entre BQP et $\mathcal{NP}$ est encore mal compris.

$$\mathcal{P} \subseteq \mathcal{NP} \subseteq \text{PSPACE} \subseteq \text{EXPTIME}$$

On sait que $\mathcal{P} \neq$ EXPTIME (au moins une de ces inclusions est stricte), mais on ne sait pas *laquelle*.

5.6.1 Problèmes $\mathcal{NP}$-intermédiaires : le théorème de Ladner

Si $\mathcal{P} \neq \mathcal{NP}$, le monde de $\mathcal{NP}$ n'est pas une simple dichotomie "facile / NP-complet".

À retenir

Théorème de Ladner (1975) : Si $\mathcal{P} \neq \mathcal{NP}$, alors il existe des problèmes dans $\mathcal{NP}$ qui ne sont **ni** dans $\mathcal{P}$, **ni** $\mathcal{NP}$-complets. Ces problèmes sont dits **$\mathcal{NP}$-intermédiaires**.

À vous de jouer !

Quels problèmes pourraient être $\mathcal{NP}$-intermédiaires ? Voici les candidats les plus sérieux :

- **Factorisation d'entiers** : décomposer un nombre en facteurs premiers. Personne n'a trouvé d'algorithme polynomial, mais personne n'a prouvé que c'est $\mathcal{NP}$-complet non plus. (La sécurité de RSA repose sur sa difficulté supposée.)
- **Isomorphisme de graphes** : deux graphes sont-ils "les mêmes" à un renommage des sommets près ? En 2015, László Babai a montré un algorithme quasi-polynomial, ce qui renforce l'idée que ce problème n'est pas $\mathcal{NP}$-complet.

5.6.2 La hiérarchie polynomiale

On peut aller encore plus loin dans la stratification de $\mathcal{NP}$.

À retenir

La **hiérarchie polynomiale** (PH) est une tour infinie de classes de complexité qui généralise $\mathcal{NP}$ et co-$\mathcal{NP}$:

- $\Sigma_0^P = \Pi_0^P = \mathcal{P}$
- $\Sigma_1^P = \mathcal{NP}$ ("$\exists$ un certificat polynomial tel que…")
- $\Pi_1^P = \text{co-}\mathcal{NP}$ ("$\forall$ les certificats polynomiaux, on a…")
- Σ_2^P ("$\exists \ldots \forall \ldots$ tel que…") deux alternances de quantificateurs

- Et ainsi de suite...

$\mathrm{PH} = \bigcup_{k \geq 0} \Sigma_k^P$

Pause café

L'idée est simple : $\mathcal{NP}$ correspond à un *seul* quantificateur existentiel ("il existe un certificat"). Que se passe-t-il si on empile les quantificateurs ? "Il existe un x tel que pour tout y, il existe un z tel que..." Chaque niveau ajoute un quantificateur. Plus on monte, plus on est expressif.
Conséquence spectaculaire : si $\mathcal{P} = \mathcal{NP}$, alors **toute** la hiérarchie polynomiale s'effondre sur $\mathcal{P}$. C'est l'un des arguments les plus convaincants *contre* $\mathcal{P} = \mathcal{NP}$: un tel effondrement semble extrêmement improbable.

5.7 L'état de la question : où en est-on ?

À retenir

Ce qu'on sait :

- $\mathcal{P} \subseteq \mathcal{NP}$
- Si un seul problème $\mathcal{NP}$-complet est dans $\mathcal{P}$, alors $\mathcal{P} = \mathcal{NP}$
- Des milliers de problèmes sont $\mathcal{NP}$-complets
- Personne n'a trouvé d'algorithme polynomial pour aucun d'entre eux

Ce qu'on ne sait pas :

- Si $\mathcal{P} = \mathcal{NP}$ ou $\mathcal{P} \neq \mathcal{NP}$
- Pourquoi c'est si difficile à prouver dans un sens ou dans l'autre

Un peu d'histoire

De nombreuses tentatives de preuves ont été publiées, tant pour $\mathcal{P} = \mathcal{NP}$ que pour $\mathcal{P} \neq \mathcal{NP}$. Toutes ont été réfutées. En 2010, Vinay Deolalikar de HP Labs a soumis une preuve de 100 pages que $\mathcal{P} \neq \mathcal{NP}$. Elle a été analysée par la communauté en quelques semaines et des erreurs fondamentales ont été trouvées.
Le consensus de la communauté (pas unanime, mais très large) est que $\mathcal{P} \neq \mathcal{NP}$. Mais le consensus ne fait pas une preuve. En mathématiques, seule la preuve compte.
Factorisation et cryptographie post-quantique. En 1994, Peter Shor a montré qu'un **ordinateur quantique** pourrait factoriser des entiers en temps polynomial brisant ainsi la sécurité de RSA. Ce résultat a déclenché une course mondiale : en 2024, le NIST (l'organisme de standardisation américain) a officiellement standardisé de

nouveaux algorithmes cryptographiques dits “post-quantiques”, conçus pour résister même à un futur ordinateur quantique.
Cela illustre comment un résultat de complexité (même *théorique* et *conditionnel* à l’existence d’ordinateurs quantiques suffisamment puissants) peut avoir des conséquences concrètes majeures sur la sécurité informatique mondiale.

Pause café

Le décompte des preuves ratées. Le blog de Scott Aaronson (informaticien à l’Université du Texas) tient un décompte informel des preuves soumises de $\mathcal{P} = \mathcal{NP}$ ou $\mathcal{P} \neq \mathcal{NP}$. Le compteur dépasse allègrement la centaine. Toutes ont été réfutées. **Conseil :** si vous pensez avoir une preuve, vérifiez d’abord qu’elle ne prouve pas *aussi* que $1 = 2$. 😉

Pause café

Pour finir sur une note philosophique : le problème P vs NP touche à quelque chose de profond sur la nature de la créativité.
Si $\mathcal{P} = \mathcal{NP}$, cela signifierait que la **découverte** (trouver une preuve, une solution, une œuvre d’art) n’est pas fondamentalement plus difficile que la **vérification** (vérifier qu’une preuve est correcte, qu’une solution marche, qu’une œuvre est belle). La plupart des gens mathématiciens, artistes, philosophes pensent intuitivement que créer est plus difficile que juger. Mais transformer cette intuition en une preuve mathématique rigoureuse ?

5.8 Quiz final

Quiz rapide — Quiz final du cours

1. Qu’est-ce qu’un problème $\mathcal{NP}$-complet ?
   a) Un problème impossible à résoudre
   b) Un problème dans $\mathcal{NP}$ auquel tout problème de $\mathcal{NP}$ se réduit
   c) Un problème qui n’est pas dans $\mathcal{P}$
   d) Un problème indécidable
2. Si demain quelqu’un trouve un algorithme polynomial pour SAT :
   a) SAT sort de $\mathcal{NP}$
   b) $\mathcal{P} = \mathcal{NP}$
   c) SAT n’est plus $\mathcal{NP}$-complet
   d) Rien de spécial
3. La cryptographie RSA repose sur la difficulté de :

a) Trier des nombres
b) Factoriser de grands nombres
c) Résoudre des Sudoku
d) Colorier des graphes

4. Quelle est la meilleure stratégie face à un problème $\mathcal{NP}$-complet en pratique ?

a) Abandonner
b) Utiliser un algorithme exact et attendre
c) Chercher des approximations ou des heuristiques
d) Acheter un plus gros ordinateur

5. Le problème de l'arrêt est :

a) $\mathcal{NP}$-complet
b) Dans $\mathcal{P}$
c) Indécidable
d) Dans $\mathcal{NP}$ mais pas dans $\mathcal{P}$

Réponse

1. **b)** C'est la définition ! Un problème $\mathcal{NP}$-complet est dans $\mathcal{NP}$ et tout problème de $\mathcal{NP}$ s'y réduit. Ce n'est pas "impossible" (il est décidable) ni forcément "hors de $\mathcal{P}$" (on ne sait pas).

2. **b)** Puisque SAT est $\mathcal{NP}$-complet, tout problème $\mathcal{NP}$ se réduit à SAT. Si SAT est dans $\mathcal{P}$, alors tout $\mathcal{NP}$ est dans $\mathcal{P}$.

3. **b)** RSA repose sur le fait qu'il est facile de multiplier deux grands nombres premiers, mais (apparemment) très difficile de retrouver les facteurs à partir du produit. Ce problème est dans $\mathcal{NP}$ (vérifier est facile : il suffit de multiplier) mais on ne connaît pas d'algorithme polynomial pour le résoudre.

4. **c)** En pratique, on utilise des algorithmes d'approximation, des heuristiques, ou des solveurs SAT. Acheter un plus gros ordinateur ne résout pas un problème exponentiel (doubler la puissance ne fait gagner qu'une unité sur n pour 2^n).

5. **c)** Le problème de l'arrêt est *indécidable* : aucune machine de Turing ne peut le résoudre. Il est "au-dessus" de $\mathcal{NP}$ dans la hiérarchie ce n'est même pas un problème de complexité, c'est un problème de calculabilité.

5.9 Exercices

5.9.0.0.1 Exercice 1 — Le labyrinthe simple (P). On considère un labyrinthe fini $n \times n$ représenté comme un graphe $G = (V, E)$ où V est l'ensemble des cases libres et

E les arêtes entre cases adjacentes. Soit S le départ et F l'arrivée.

a) Montrez que le problème "existe-t-il un chemin de S à F dans G ?" est dans $\mathcal{P}$. Proposez un algorithme polynomial et analysez sa complexité.

b) Quel est le certificat pour montrer que ce problème est dans $\mathcal{NP}$? Vérifiez que sa vérification est polynomiale.

c) Montrez que le problème "quel est le **plus court** chemin de S à F ?" est aussi dans $\mathcal{P}$.

5.9.0.0.2 Exercice 2 Le labyrinthe à clés ($\mathcal{NP}$). On ajoute maintenant des **portes** (arêtes bloquées) et des **clés** (objets à ramasser sur certaines cases). Une porte ne peut être franchie que si la clé correspondante a été collectée **auparavant** sur le chemin.

a) Proposez un certificat pour montrer que ce problème est dans $\mathcal{NP}$. Montrez que sa vérification est polynomiale.

b) Esquissez une réduction depuis le problème du chemin hamiltonien (Ham-Path) vers ce problème de labyrinthe à clés. (Indication : chaque sommet du graphe original devient une case avec une clé unique, et l'arrivée est protégée par une porte nécessitant *toutes* les clés.)

c) Que peut-on conclure sur la complexité du labyrinthe à clés ?

5.9.0.0.3 Exercice 3 Réductions en chaîne. On sait que SAT est $\mathcal{NP}$-complet. On sait que 3-Sat $\leq_p$ Clique $\leq_p$ Ensemble Indépendant. On sait aussi que 3-Sat $\leq_p$ 3-Coloration.

a) En déduire que Ensemble Indépendant et 3-Coloration sont $\mathcal{NP}$-complets (en supposant qu'ils sont dans $\mathcal{NP}$, ce qu'on admet).

b) Pourquoi suffit-il de montrer la réduction depuis *un seul* problème $\mathcal{NP}$-complet connu, et non depuis *tous* les problèmes de $\mathcal{NP}$?

c) Si demain on découvre un algorithme polynomial pour 3-Coloration, quelles seraient les conséquences ?

5.9.0.0.4 Exercice 4 Réflexion ouverte. Le problème de la factorisation (décomposer un entier en facteurs premiers) est dans $\mathcal{NP}$ mais n'est pas connu comme $\mathcal{NP}$-complet.

a) Quel serait le certificat ? Est-il vérifiable en temps polynomial ?

b) Si quelqu'un prouvait que la factorisation est $\mathcal{NP}$-complète, quelles seraient les conséquences pour la cryptographie RSA ?

c) Si quelqu'un trouvait un algorithme polynomial pour la factorisation, cela prouverait-il que $\mathcal{P} = \mathcal{NP}$? Pourquoi ?

5.10 Conclusion du cours

Nous avons parcouru un long chemin ensemble. Récapitulons :

1. **Chapitre 1 — Introduction.** Problèmes, algorithmes, et la question fondamentale : tous les problèmes ont-ils un algorithme ?
2. **Chapitre 2 — Machines de Turing.** Le modèle de calcul universel aussi puissant que n'importe quel ordinateur.
3. **Chapitre 3 — Décidabilité et indécidabilité.** La preuve qu'il existe des problèmes qu'aucun algorithme ne peut résoudre. Le problème de l'arrêt. Les réductions.
4. **Chapitre 4 — Classes de complexité.** Parmi les problèmes décidables, certains sont "faciles" ($\mathcal{P}$) et d'autres semblent "difficiles" ($\mathcal{NP}$).
5. **Chapitre 5 — P vs NP.** La question à un million de dollars. Les problèmes $\mathcal{NP}$-complets. SAT. Les stratégies pratiques.

Question de réflexion

Voici une dernière question ouverte, pour la route :
On a vu que le non-déterminisme permet de "deviner" la bonne solution. Les ordinateurs quantiques, avec la superposition d'états, permettent d'explorer plusieurs chemins "en parallèle". Est-ce que les ordinateurs quantiques pourraient résoudre les problèmes $\mathcal{NP}$-complets en temps polynomial ?
(Spoiler : probablement pas. Mais c'est un sujet de recherche actif, et la réponse définitive n'est pas encore connue.)

Pause café

Félicitations ! Vous êtes arrivé au bout de ce cours. Vous savez maintenant des choses que la grande majorité des gens ignorent :

- Qu'il existe des limites *fondamentales* à ce que les ordinateurs peuvent faire.
- Que ces limites ne sont pas technologiques mais *mathématiques*.
- Qu'il existe une question ouverte qui vaut un million de dollars et que personne au monde n'a réussi à résoudre depuis plus de 50 ans.

Si vous avez compris tout ça, vous pouvez vous considérer comme un(e) informaticien(ne) théorique en herbe. Et si un jour vous résolvez P vs NP… n'oubliez pas de citer ce cours dans votre publication. 😊